Pedro Henrique Gomes

Análise Experimental da Inferência SepConv em C++ com Ênfase em Eficiência Computacional e Uso de VRAM

São Paulo
2026

Pedro Henrique Gomes

Análise Experimental da Inferência SepConv em C++ com Ênfase em Eficiência Computacional e Uso de VRAM

Trabalho de Conclusão de Curso apresentado ao SENAC Santo Amaro como requisito parcial para aprovação na disciplina de TCC2 do curso de Bacharelado em Ciência da Computação.

SENAC – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Curso de Bacharelado em Ciência da Computação

Orientador: Prof. Afonso Brandão

São Paulo
2026

Resumo

A disseminação de monitores com taxas de atualização de 60 Hz a 144 Hz evidenciou a limitação temporal de conteúdos audiovisuais capturados a 24 FPS ou 30 FPS. A Interpolação de Quadros de Vídeo (VFI – *Video Frame Interpolation*) sintetiza quadros intermediários para aumentar a fluidez, mas modelos de aprendizado profundo executados em *frameworks* de alto nível, como o PyTorch, demandam grande quantidade de memória de vídeo (VRAM). As Convoluções Adaptativas Separáveis (SepConv) estimam, para cada pixel, um par de filtros unidimensionais de N coeficientes em vez de um filtro bidimensional de N^2 coeficientes, o que reduz a memória ocupada pelos filtros (cerca de 1,6 GB contra 40 GB em 1080p, com $N = 51$), embora a síntese continue realizando N^2 multiplicações por pixel. Este trabalho avaliou experimentalmente se um motor de inferência nativo em C++ e CUDA para o modelo SepConv++ reduz a latência e o pico de VRAM em uma GPU de consumo de 8 GB, em comparação com a implementação original em Python/PyTorch. O motor integra a biblioteca OpenCV para entrada e saída, a API C++ do ONNX Runtime com *I/O binding* para a inferência da rede e um *kernel* CUDA próprio para a convolução separável. As duas implementações interpolaram com fator 2× um trecho de 61 quadros de vídeo em 720p, 1080p, 1440p e 2160p, com medição da latência por quadro, do pico de VRAM via NVML e da fidelidade visual por PSNR, SSIM e VMAF. Em 720p e 1080p, o motor C++ reduziu a latência em cerca de 24% (305,5 ms contra 404,9 ms por quadro em 1080p) e produziu quadros com a mesma fidelidade da implementação original (34,10 dB de PSNR em 1080p nas duas), mas consumiu cerca de 45% mais VRAM (6.648 MiB contra 4.585 MiB). A partir de 1440p, a VRAM se esgotou: o motor ficou 5,8 vezes mais lento que a implementação original e falhou em 2160p. Uma segunda versão, com processamento em blocos, fez o pico de VRAM depender do tamanho do bloco, e não da resolução, mas aumentou a latência em 720p e 1080p. Uma terceira versão desativou o padrão de memória do ONNX Runtime, principal fonte do consumo excedente, e passou a executar a rede em meia precisão (FP16). Com o quadro inteiro em FP16, o motor reduziu a latência em 55% (183,0 ms contra 406,3 ms em 1080p) e o pico de VRAM em 33% em relação à implementação original, com a mesma fidelidade no vídeo de teste e nos conjuntos Middlebury e Vimeo-90K; em 1440p, foi 4,1 vezes mais rápido sem saturar a VRAM e, em 2160p, com blocos, sintetizou cada quadro em 1,19 s, contra 66,6 s. Com a implementação original também em FP16, o motor manteve a vantagem em 1080p, com 39,8% menos latência e 20% menos VRAM. Conclui-se que a transposição da inferência para C++ reduz a latência sem perda de qualidade, mas que a redução de VRAM não decorre da troca de linguagem ou de *framework*: ela depende de decisões explícitas de gerenciamento de memória e de precisão numérica, que o motor nativo torna controláveis.

Palavras-chave: Interpolação de Quadros de Vídeo. Convoluções Adaptativas Separáveis. C++. CUDA. Otimização de VRAM. SepConv.

Abstract

The spread of displays with refresh rates from 60 Hz to 144 Hz has highlighted the temporal limitation of audiovisual content captured at 24 FPS or 30 FPS. Video Frame Interpolation (VFI) synthesizes intermediate frames to increase smoothness, but deep learning models executed on high-level frameworks, such as PyTorch, demand large amounts of video memory (VRAM). Adaptive Separable Convolutions (SepConv) estimate, for each pixel, a pair of one-dimensional filters with N coefficients instead of a two-dimensional filter with N^2 coefficients, which reduces the memory occupied by the filters (about 1.6 GB versus 40 GB at 1080p, with $N = 51$), although the synthesis still performs N^2 multiplications per pixel. This work experimentally evaluated whether a native C++ and CUDA inference engine for the SepConv++ model reduces latency and peak VRAM usage on an 8 GB consumer GPU, compared with the original Python/PyTorch implementation. The engine integrates the OpenCV library for input and output, the ONNX Runtime C++ API with I/O binding for network inference, and a custom CUDA kernel for the separable convolution. Both implementations interpolated a 61-frame video excerpt with a 2× factor at 720p, 1080p, 1440p and 2160p, measuring per-frame latency, peak VRAM via NVML, and visual fidelity through PSNR, SSIM and VMAF. At 720p and 1080p, the C++ engine reduced latency by about 24% (305.5 ms versus 404.9 ms per frame at 1080p) and produced frames with the same fidelity as the original implementation (34.10 dB PSNR at 1080p for both), but consumed about 45% more VRAM (6,648 MiB versus 4,585 MiB). From 1440p onwards, VRAM was exhausted: the engine became 5.8 times slower than the original implementation and failed at 2160p. A second version, with tiled processing, made peak VRAM depend on the tile size rather than on the resolution, but increased latency at 720p and 1080p. A third version disabled the ONNX Runtime memory pattern, the main source of the excess consumption, and ran the network in half precision (FP16). With the whole frame in FP16, the engine reduced latency by 55% (183.0 ms versus 406.3 ms at 1080p) and peak VRAM by 33% compared with the original implementation, with the same fidelity on the test video and on the Middlebury and Vimeo-90K datasets; at 1440p, it was 4.1 times faster without saturating VRAM and, at 2160p, using tiles, it synthesized each frame in 1.19 s, versus 66.6 s. With the original implementation also in FP16, the engine kept its advantage at 1080p, with 39.8% lower latency and 20% less VRAM. It is concluded that moving inference to C++ reduces latency without loss of quality, but that VRAM reduction does not follow from the change of language or framework: it depends on explicit memory-management and numerical-precision decisions, which the native engine makes controllable.

Keywords: Video Frame Interpolation. Adaptive Separable Convolutions. C++. CUDA. VRAM Optimization. SepConv.

Lista de Abreviaturas e Siglas

A lista a seguir reúne, em ordem alfabética, as abreviaturas e siglas utilizadas neste trabalho, acompanhadas de seus respectivos significados.

Abreviatura/Sigla	**Significado**
ABNT	Associação Brasileira de Normas Técnicas
ALU	*Arithmetic Logic Unit* (Unidade Lógica e Aritmética)
API	*Application Programming Interface* (Interface de Programação de Aplicações)
CLI	*Command-Line Interface* (Interface de Linha de Comando)
CPU	*Central Processing Unit* (Unidade Central de Processamento)
CUDA	*Compute Unified Device Architecture*
EP	*Execution Provider* (Provedor de Execução do ONNX Runtime)
FFV1	*FFmpeg Video Codec 1* (codec de vídeo sem perdas)
FLOPs	*Floating Point Operations* (Operações de Ponto Flutuante)
FP16	Ponto flutuante de 16 bits (meia precisão)
FPS	*Frames Per Second* (Quadros por Segundo)
GPGPU	*General-Purpose Computing on Graphics Processing Units*
GPU	*Graphics Processing Unit* (Unidade de Processamento Gráfico)
I/O	*Input/Output* (Entrada e Saída)
MiB	*Mebibyte* (2^{20} bytes)
MSE	*Mean Squared Error* (Erro Quadrático Médio)
NVML	*NVIDIA Management Library*
ONNX	*Open Neural Network Exchange*
PCIe	*Peripheral Component Interconnect Express*
PSNR	*Peak Signal-to-Noise Ratio*
RAFT	*Recurrent All-Pairs Field Transforms*
RAM	*Random Access Memory*
SIMT	*Single Instruction, Multiple Threads*
SM	*Streaming Multiprocessor*
SSIM	*Structural Similarity Index*
TCC	Trabalho de Conclusão de Curso
TF32	*TensorFloat-32*
VFI	*Video Frame Interpolation* (Interpolação de Quadros de Vídeo)
VMAF	*Video Multimethod Assessment Fusion*
VRAM	*Video Random Access Memory*

Sumário

Sumário **5**

1 INTRODUÇÃO **8**
1.1 Contexto **9**
1.2 Formulação do problema **10**
1.3 Objetivos e hipótese **11**
1.3.1 Objetivo geral 11
1.3.2 Objetivos específicos 11
1.3.3 Hipótese e critérios de validação 11
1.4 Justificativa **12**
1.5 Organização do trabalho **13**

2 REVISÃO BIBLIOGRÁFICA **14**
2.1 Metodologia da pesquisa bibliográfica **14**
2.1.1 Período de abrangência 14
2.1.2 Critérios de inclusão 15
2.1.3 Critérios de exclusão 15
2.2 Fundamentos **16**
2.2.1 Definição de Imagem Digital 16
2.2.2 Filtragem Espacial e Convolução Adaptativa 18
2.2.3 Fluxo Óptico Explícito vs. Kernels de Movimento Implícito 21
2.2.4 Eficiência via Convoluções Separáveis 22
2.2.5 Implementação de Baixo Nível e I/O de Vídeo 24
2.2.6 Inferência Desacoplada via ONNX Runtime 25
2.2.7 Arquitetura de Decomposição e Garantia de Separabilidade 25
2.2.8 Arquitetura de GPU e o Gargalo de Memória (*Memory Wall*) 26
2.2.9 Modelo de Execução Paralela SIMT 27
2.2.10 Processamento de Baixo Nível e a Plataforma CUDA 28
2.2.11 Métricas de Avaliação Objetiva de Qualidade 29
2.2.12 Mensuração de Desempenho Computacional 31
2.3 Trabalhos relacionados **32**
2.3.1 Trabalho 1: Revisiting Adaptive Convolutions for Video Frame Interpolation 32
2.3.2 Trabalho 2: Motion-Aware Video Frame Interpolation 32
2.3.3 Trabalho 3: RAFT: Recurrent All-Pairs Field Transforms for Optical Flow . 33

3 DESENVOLVIMENTO **35**

3.1 Solução Proposta e Arquitetura . . . 35
3.1.1 Modelo utilizado . . . 35
3.1.2 Fluxo de dados . . . 35
3.2 Exportação do Modelo e Grafo ONNX . . . 36
3.2.1 Correção do grafo para resoluções arbitrárias . . . 38
3.3 Implementação do Motor de Inferência . . . 38
3.3.1 Integração com o ONNX Runtime . . . 39
3.3.2 *Kernel* de convolução separável . . . 39
3.3.3 Instrumentação . . . 40
3.3.4 Equivalência com a implementação original . . . 40
3.4 Processamento em blocos (segunda versão do motor) . . . 40
3.4.1 Escolha da margem . . . 41
3.5 Alocador de memória e meia precisão (terceira versão do motor) . . . 42
3.5.1 Configuração do alocador do ONNX Runtime . . . 43
3.5.2 Inferência em meia precisão (FP16) . . . 44

4 METODOLOGIA . . . 45
4.1 Delineamento e condições experimentais . . . 45
4.2 Ambiente Experimental . . . 46
4.3 Conjunto de avaliação . . . 47
4.4 Procedimento de medição . . . 47
4.5 Métricas . . . 48
4.6 Critérios de decisão e análise estatística . . . 49
4.7 Riscos e mitigações . . . 49
4.8 Cronograma de trabalho . . . 51

5 RESULTADOS E DISCUSSÃO . . . 52
5.1 Latência . . . 52
5.2 Consumo de VRAM . . . 54
5.3 Fidelidade visual . . . 55
5.4 Segunda versão: processamento em blocos . . . 58
5.5 Terceira versão: alocador ajustado e meia precisão . . . 62
5.6 Forma de execução e precisão: as quatro combinações . . . 65
5.7 Conjuntos de referência . . . 67
5.8 Discussão frente à hipótese . . . 68

6 CONCLUSÃO . . . 72
6.1 Limitações . . . 73
6.2 Trabalhos futuros . . . 74

REFERÊNCIAS 75

1 Introdução

Historicamente, os sistemas de captura e exibição de imagens em movimento foram limitados pela capacidade mecânica e elétrica dos dispositivos da época. Durante a consolidação do cinema sonoro no início do século XX, a taxa de aproximadamente 24 quadros por segundo (FPS) tornou-se um padrão industrial por representar um equilíbrio entre fluidez perceptiva e custo de armazenamento e reprodução do filme físico (BORDWELL; THOMPSON; SMITH, 2016). Taxas nessa faixa passaram a ser suficientes para produzir a percepção de movimento contínuo ao sistema visual humano, reduzindo a percepção de descontinuidade temporal durante a projeção (WATKINSON, 2009).

Com a evolução tecnológica dos dispositivos de exibição, monitores e televisores modernos passaram a operar em frequências significativamente superiores, variando entre 60 Hz e 144 Hz. Entretanto, grande parte do conteúdo audiovisual disponível ainda permanece limitada às taxas tradicionais de captura, como 24 FPS ou 30 FPS, criando uma discrepância entre a capacidade de atualização do hardware atual e a quantidade de informação temporal presente no vídeo original. Essa diferença entre a taxa de amostragem temporal do conteúdo original e a frequência de atualização do dispositivo de exibição resulta no fenômeno conhecido como subamostragem temporal. De acordo com Watkinson (2009) e Szeliski (2022), a incapacidade de fornecer amostras intermediárias suficientes durante movimentos rápidos reduz a percepção de continuidade temporal pelo sistema visual humano, induzindo a artefatos perceptuais como tremulação de movimento (*motion judder*), borramento dinâmico e quebra na percepção de fluidez contínua.

Nesse contexto, a Interpolação de Quadros de Vídeo (VFI – *Video Frame Interpolation*) surge como uma técnica essencial para sintetizar quadros intermediários artificialmente, aumentando a fluidez temporal do conteúdo exibido. Tradicionalmente, a literatura divide as abordagens de VFI em métodos baseados em fluxo óptico, como o modelo RAFT (TEED; DENG, 2020), e métodos baseados em *kernels*. Embora modelos de aprendizado profundo tenham atingido patamares elevados de precisão, eles frequentemente demandam recursos computacionais extensivos e dependem de *frameworks* pesados, o que limita sua aplicação em cenários de hardware restrito ou escassez de memória RAM e VRAM.

Para mitigar a alta demanda de recursos computacionais — representada pelo elevado volume de operações de ponto flutuante (FLOPs), pelo esgotamento do armazenamento de tensores na memória de vídeo (VRAM) e pela alta latência de execução —, a literatura recente busca alternativas mais eficientes aos modelos tradicionais de fluxo óptico. Em cenários de hardware restrito, típicos de computadores pessoais e dispositivos de consumo doméstico dotados de GPUs comerciais limitadas a 8 GB de VRAM, a execução de modelos densos em *frameworks* de alto nível (como PyTorch) torna-se inviável em altas resoluções. Nesse contexto, Niklaus, Mai e Liu (2017) propuseram as Convoluções Adaptativas Separá-

veis, nas quais a rede neural estima, para cada pixel, um par de filtros unidimensionais de N coeficientes em vez de um *kernel* bidimensional de $N \times N$ coeficientes. Essa formulação reduz de N^2 para $2N$ a quantidade de coeficientes estimados e armazenados por pixel e, portanto, a memória ocupada pelos tensores de filtros. A implementação dessa etapa de inferência em linguagem de baixo nível (C++/CUDA) possibilita um controle granular sobre o gerenciamento de memória gráfica, permitindo isolar a eficiência do algoritmo e testar sua viabilidade em computadores comuns.

Diante desse cenário, este trabalho investiga experimentalmente se a execução nativa da inferência do modelo SepConv em C++ e CUDA reduz a latência e o consumo de VRAM em uma GPU de consumo de 8 GB, em comparação com a implementação original em Python/PyTorch, sem degradar a fidelidade visual dos quadros sintetizados. A questão de pesquisa, os objetivos e a hipótese são formalizados nas Seções 1.2 e 1.3.

1.1 Contexto

Diferente de métodos simplificados de amostragem de quadros, como a repetição ou a mistura linear de quadros vizinhos, a VFI busca sintetizar um novo quadro no instante $t + 0{,}5$ por meio de uma combinação ponderada de informações espaciais e temporais das imagens adjacentes. Esse processo considera o contexto do movimento e a correlação entre pixels (NIKLAUS; MAI; LIU, 2017). Ao elevar a taxa de quadros por segundo, a técnica objetiva atenuar artefatos de movimento na percepção humana. A Figura 1 mostra uma sequência real com movimento: as bolas e as mãos mudam de posição entre os quadros, e o objetivo é obter o quadro do meio.

Figura 1 – Interpolação de quadros: a partir dos quadros nos instantes t e $t+1$, sintetiza-se o quadro no instante $t + 0{,}5$. O quadro central é o quadro real, usado como referência.

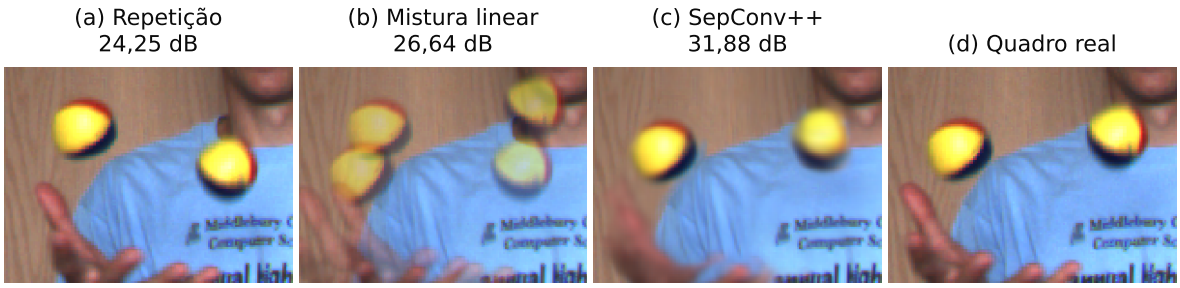


Fonte: Elaborado pelo autor (2026), a partir da sequência Beanbags do conjunto Middlebury (BAKER et al., 2011).

A Figura 2 compara essas abordagens na região de maior movimento da mesma cena. A repetição mantém a bola na posição anterior; a mistura linear produz duas cópias

semitransparentes, o efeito conhecido como *ghosting*; e a interpolação pelo SepConv++ reconstrói a bola em uma única posição, próxima à real, embora com algum borramento. Na imagem inteira, o PSNR em relação ao quadro real é de 24,25 dB, 26,64 dB e 31,88 dB, respectivamente.

Figura 2 – Comparação entre métodos de obtenção do quadro intermediário, com o PSNR da imagem inteira em relação ao quadro real.



Fonte: Elaborado pelo autor (2026), a partir da sequência Beanbags do conjunto Middlebury (BAKER et al., 2011).

Entretanto, a implantação prática desses algoritmos enfrenta barreiras atreladas ao custo computacional espacial e aritmético. Modelos de estado da arte demandam alto armazenamento de tensores intermediários na memória de vídeo (VRAM), limitando frequentemente sua execução a hardwares de alto desempenho (BAO et al., 2019). A transposição de modelos matemáticos para *pipelines* de inferência otimizados surge como um caminho científico relevante para avaliar os reais gargalos de desempenho em sistemas com memória de consumo limitada.

1.2 Formulação do problema

Apesar da elevada precisão atingida por modelos modernos de Interpolação de Quadros de Vídeo (VFI), a dependência de *frameworks* de alto nível (como PyTorch) e a alta demanda por memória de vídeo impedem sua execução fluida e eficiente em computadores pessoais munidos de GPUs comerciais. Diante dessa limitação física e arquitetural, formula-se a seguinte questão de pesquisa:

> *Em que medida a implementação de um motor de inferência nativo em C++ e CUDA para o modelo SepConv, que estima filtros separáveis com $2N$ coeficientes por pixel, é capaz de reduzir a latência de processamento e o consumo de VRAM em GPUs de consumo de 8 GB, mantendo a fidelidade visual da implementação original?*

1.3 Objetivos e hipótese

Esta seção delimita o escopo técnico da pesquisa, focando na avaliação de desempenho sob restrições físicas de VRAM em hardware de consumo.

1.3.1 Objetivo geral

Avaliar experimentalmente o desempenho computacional da inferência do modelo SepConv em GPU, utilizando uma implementação nativa em C++ e CUDA, com ênfase na latência de execução e no pico de consumo de memória VRAM em hardware de consumo.

1.3.2 Objetivos específicos

1. Desenvolver um módulo de Entrada e Saída (I/O) em C++ e OpenCV para extração e linearização de quadros de vídeo na memória principal (Host) e transferência contígua para a memória gráfica (Device);

2. Integrar a API C++ do *ONNX Runtime* (operando via *CUDA Execution Provider*) para carregar o modelo SepConv exportado e inferir na VRAM os filtros unidimensionais ($1D$) de cada pixel;

3. Implementar um *kernel* CUDA próprio para a síntese do quadro intermediário a partir dos filtros separáveis, mantendo os dados inteiramente na VRAM;

4. Desenvolver um ambiente de testes comparativo entre a implementação nativa proposta (C++/CUDA) e a *baseline* original do modelo SepConv em Python/PyTorch;

5. Quantificar a eficiência computacional por meio do monitoramento da latência por quadro (em milissegundos, via `cudaEventRecord`), da taxa de quadros por segundo (FPS) e do pico de ocupação de VRAM (via biblioteca NVML) para diferentes resoluções de vídeo;

6. Avaliar a fidelidade visual dos quadros sintetizados por meio das métricas objetivas *Peak Signal-to-Noise Ratio* (PSNR), *Structural Similarity Index* (SSIM) e *Video Multimethod Assessment Fusion* (VMAF), comparando-os aos quadros reais de um trecho de vídeo de referência.

1.3.3 Hipótese e critérios de validação

Postula-se como **hipótese científica** que um *pipeline* de inferência nativo em C++ e CUDA para o modelo SepConv, desacoplado do *runtime* do Python, reduz a latência por quadro e o pico de consumo de VRAM em comparação com a implementação original em Python/PyTorch, sem degradar a fidelidade visual. A hipótese é desdobrada em

três critérios, verificados separadamente em cada resolução (o procedimento estatístico é detalhado na Seção 4.6):

H1 **Latência**: o motor C++ reduz a latência média por quadro em pelo menos 10% em relação à *baseline*, com diferença significativa no teste t de Welch unilateral ao nível $\alpha = 0{,}05$;

H2 **VRAM**: o pico de VRAM do motor C++ é no máximo 90% do pico da *baseline* (redução mínima de 10%) e permanece abaixo de 97% da capacidade da GPU, sem recorrer à memória principal;

H3 **Fidelidade**: a diferença média entre as implementações não excede 0,1 dB de PSNR, 0,001 de SSIM e 0,5 ponto de VMAF.

Neste trabalho, considera-se **alta definição** a resolução de 1920 × 1080 pixels (1080p). As resoluções 1440p e 2160p são avaliadas como extensão, para caracterizar o comportamento das implementações quando a VRAM se esgota.

1.4 Justificativa

A escolha pela investigação de algoritmos de interpolação baseados em Convoluções Adaptativas Separáveis justifica-se, primordialmente, pela necessidade de viabilizar a fluidez temporal em computadores pessoais com recursos de memória gráfica restritos. Em um *kernel* adaptativo bidimensional completo, a rede precisaria estimar e armazenar N^2 coeficientes por pixel; a decomposição em dois filtros unidimensionais reduz essa quantidade para $2N$. Com $N = 51$, em resolução Full HD, os filtros separáveis ocupam cerca de 1,6 GB, enquanto filtros bidimensionais exigiriam aproximadamente 40 GB (Seção 3.3.2). É essa redução do *footprint* (consumo espacial) dos tensores mantidos na VRAM que torna o modelo executável em hardware comercial.

A delimitação experimental em GPUs de consumo dotadas de 8 GB de VRAM fundamenta-se na representatividade desse perfil de hardware entre usuários domésticos. Segundo dados da *Steam Hardware & Software Survey* (Valve Corporation, 2026), placas gráficas com 8 GB de memória representam aproximadamente 26% das configurações ativas no mercado consumidor. Embora modelos corporativos e GPUs de alto desempenho (com 16 GB a 80 GB de VRAM) tenham registrado avanço, a faixa de 8 GB constitui o principal gargalo físico para a execução de modelos densos de inteligência artificial fora de *data centers*.

Sob a perspectiva da Ciência da Computação e da Engenharia de Software, este trabalho justifica-se pela análise do custo imposto por *frameworks* de alto nível (como Python e PyTorch). Embora adequados para as fases de treinamento e prototipagem, esses ambientes acrescentam camadas entre a aplicação e a GPU — como o despacho de operações pelo

interpretador e as alocações dinâmicas gerenciadas pelo alocador do *framework* —, cujo efeito sobre a latência e a memória não é conhecido de antemão. Supõe-se que, ao transpor a etapa de inferência do modelo SepConv para um ambiente nativo em C++, integrado à API do ONNX Runtime e a um *kernel* próprio em CUDA, seja possível exercer um controle explícito sobre a alocação de memória na VRAM. Essa suposição é tratada como hipótese e verificada experimentalmente no Capítulo 5.

Em suma, a justificativa do estudo reside na verificação prática de se o gerenciamento granular de hardware, aliado à otimização algorítmica de baixo nível, permite superar limitações físicas de memória, oferecendo um ponto de equilíbrio entre a qualidade visual da interpolação e a eficiência computacional em sistemas comerciais de consumo.

1.5 Organização do trabalho

O Capítulo 2 apresenta a pesquisa bibliográfica, os fundamentos teóricos e os trabalhos relacionados. O Capítulo 3 descreve a arquitetura, a exportação do modelo para ONNX e a implementação do motor em C++ e CUDA, em três versões sucessivas. O Capítulo 4 define o ambiente, o conjunto de avaliação, o procedimento de medição e os critérios de análise. O Capítulo 5 apresenta e discute os resultados frente à hipótese, e o Capítulo 6 responde à questão de pesquisa e aponta limitações e trabalhos futuros.

2 Revisão Bibliográfica

Este capítulo descreve inicialmente o processo de pesquisa bibliográfica. Em seguida, apresenta os fundamentos necessários ao trabalho — representação de imagens, convolução adaptativa e separável, inferência via ONNX Runtime, arquitetura de GPU e métricas de avaliação — e, por fim, os trabalhos relacionados que delimitam a contribuição desta pesquisa.

2.1 Metodologia da pesquisa bibliográfica

A pesquisa bibliográfica foi conduzida por meio de buscas nas bases Google Scholar e IEEE Xplore, utilizando combinações de palavras-chave relacionadas ao tema da pesquisa. As consultas empregaram operadores lógicos para refinar os resultados, buscando trabalhos relacionados à interpolação de quadros de vídeo, convoluções separáveis, fluxo óptico e otimização de inferência em GPU.

- **Bases de dados consultadas**: Google Scholar e IEEE Xplore;
- **Palavras-chave utilizadas**:
  - ("video frame interpolation"OR "frame interpolation") AND ("convolutional neural network"OR CNN OR convolution) AND (video OR image sequence);
  - ("video frame interpolation") AND (convolution OR "adaptive convolution"OR "separable convolution");
  - ("video frame interpolation") AND ("optical flow"OR convolution OR transformer OR "deep learning");

2.1.1 Período de abrangência

Embora as buscas tenham retornado publicações de diferentes períodos, foi adotado como critério principal a seleção de trabalhos publicados a partir de 2015. Esse recorte temporal foi definido por contemplar o surgimento e a consolidação das principais técnicas modernas de aprendizado profundo aplicadas à interpolação de quadros de vídeo. Trabalhos anteriores foram mantidos apenas quando considerados referências clássicas ou fundamentais para a compreensão dos conceitos teóricos utilizados nesta pesquisa.

O processo de busca inicial retornou aproximadamente 5.000 resultados. Em uma primeira etapa de triagem (*screening*), foi realizada a leitura dos títulos dos trabalhos encontrados, resultando na seleção de aproximadamente 60 publicações potencialmente

relevantes ao tema investigado. Após a aplicação do filtro temporal, cerca de 20 artigos permaneceram para análise.

Na etapa seguinte, foi realizada a leitura dos resumos dos trabalhos selecionados, reduzindo o conjunto para aproximadamente 12 publicações. Por fim, foi conduzida uma leitura mais aprofundada dos artigos com maior aderência ao escopo da pesquisa, dos quais cerca de cinco foram efetivamente utilizados na construção do referencial teórico e três foram selecionados para compor a seção de trabalhos relacionados.

2.1.2 Critérios de inclusão

Foram considerados elegíveis os trabalhos que atendessem a pelo menos um dos seguintes critérios:

- abordagem relacionada à interpolação de quadros de vídeo (*Video Frame Interpolation*);
- utilização de convoluções adaptativas ou convoluções separáveis;
- aplicação de técnicas de fluxo óptico para síntese temporal de quadros;
- estudos envolvendo inferência acelerada por GPU;
- pesquisas relacionadas à otimização computacional, consumo de memória ou redução de latência em aplicações de visão computacional;
- publicações com relevância científica comprovada por meio de conferências ou periódicos reconhecidos na área.

2.1.3 Critérios de exclusão

Foram excluídos da análise trabalhos que apresentassem uma ou mais das seguintes características:

- baixa aderência ao problema investigado;
- foco exclusivo na etapa de treinamento de modelos neurais;
- dependência excessiva de hardware específico sem aplicabilidade ao contexto da pesquisa;
- ausência de descrição técnica suficiente para reprodução ou compreensão da metodologia proposta;
- duplicidade entre bases consultadas;

- trabalhos que não abordassem aspectos relacionados à interpolação de quadros, fluxo óptico, convoluções separáveis ou otimização de inferência.

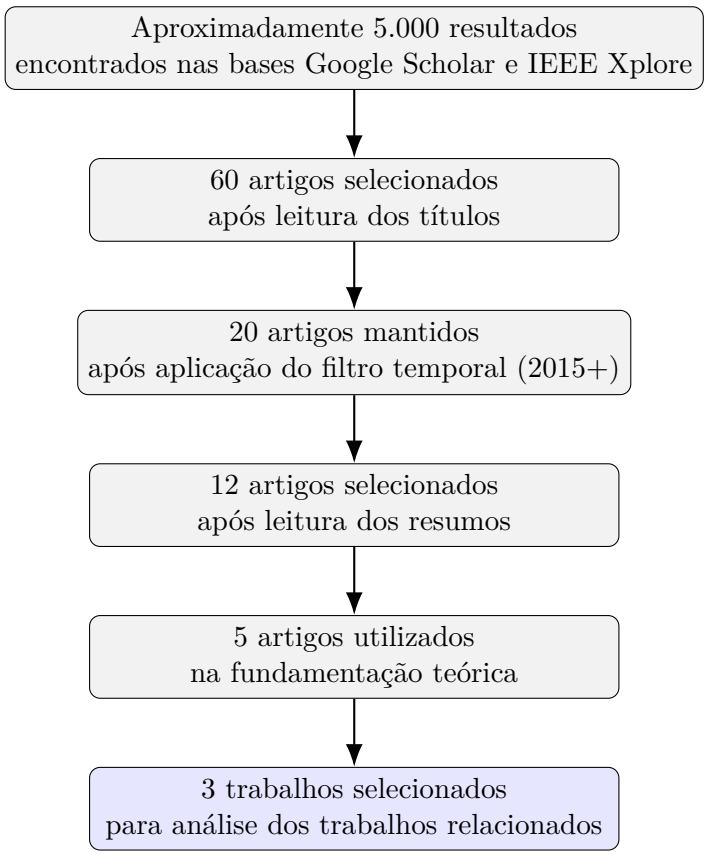


Figura 3 – Processo de seleção dos trabalhos utilizados na pesquisa bibliográfica.

Fonte: Elaborado pelo autor (2026)

A estratégia de seleção adotada permitiu reduzir progressivamente o conjunto inicial de publicações, mantendo apenas trabalhos com elevada aderência técnica ao objetivo da pesquisa e representativos do estado da arte em interpolação de quadros de vídeo e otimização de inferência em GPU.

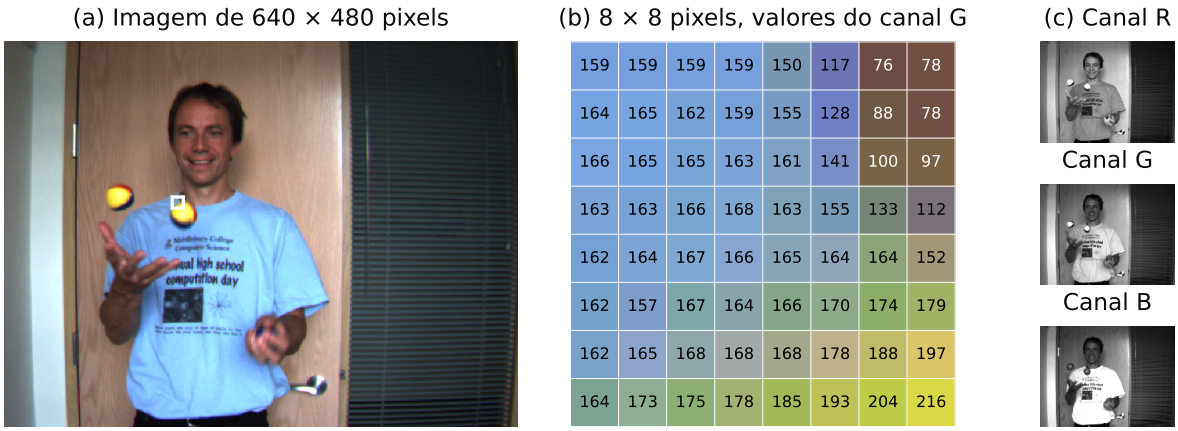
2.2 Fundamentos

2.2.1 Definição de Imagem Digital

Uma imagem digital pode ser definida como uma função bidimensional $f(x, y)$, onde x e y representam coordenadas espaciais em um plano. De acordo com Gonzalez e Woods (2009), quando os valores de x, y e a amplitude de f assumem quantidades finitas e discretas, a imagem é caracterizada como digital. Na prática computacional, essa estrutura é representada por uma matriz composta por elementos discretos denominados *pixels* (*picture elements*), que constituem a menor unidade de informação visual de uma imagem.

No modelo de cores RGB (*Red, Green, Blue*), amplamente utilizado em sistemas computacionais, cada pixel é representado por três componentes correspondentes aos canais vermelho, verde e azul. Em implementações convencionais de 8 bits por canal, cada componente assume valores no intervalo $[0, 255]$, permitindo a representação de aproximadamente 16,7 milhões de combinações de cores (SZELISKI, 2022). A Figura 4 ilustra essa representação em uma imagem real: os valores numéricos de uma pequena região e os três canais de cor, cada um deles uma matriz própria.

Figura 4 – Imagem digital como matriz de pixels: (a) imagem, com a região ampliada em destaque; (b) valores de um canal nessa região; (c) os três canais de cor.



Fonte: Elaborado pelo autor (2026), a partir da sequência Beanbags do conjunto Middlebury (BAKER et al., 2011).

A resolução de uma imagem corresponde à quantidade total de pixels que a compõem. Em aplicações de vídeo, o volume de dados cresce rapidamente com o aumento da resolução. Por exemplo, um único quadro em resolução Full HD (1920×1080) contém mais de dois milhões de pixels. Considerando três canais de cor por pixel, a manipulação simultânea de múltiplos quadros pode demandar dezenas de megabytes de memória, tornando o gerenciamento eficiente dos dados um aspecto relevante para algoritmos de processamento de vídeo e interpolação de quadros (SZELISKI, 2022).

A representação dessas matrizes em linguagem C++ impacta diretamente a performance computacional. Embora conceitualmente bidimensionais, as imagens são frequentemente organizadas na memória como vetores unidimensionais para maximizar o acesso contíguo aos dados. Nesse formato de linearização, a posição de um pixel (x, y) em uma imagem de largura W pode ser obtida pela expressão:

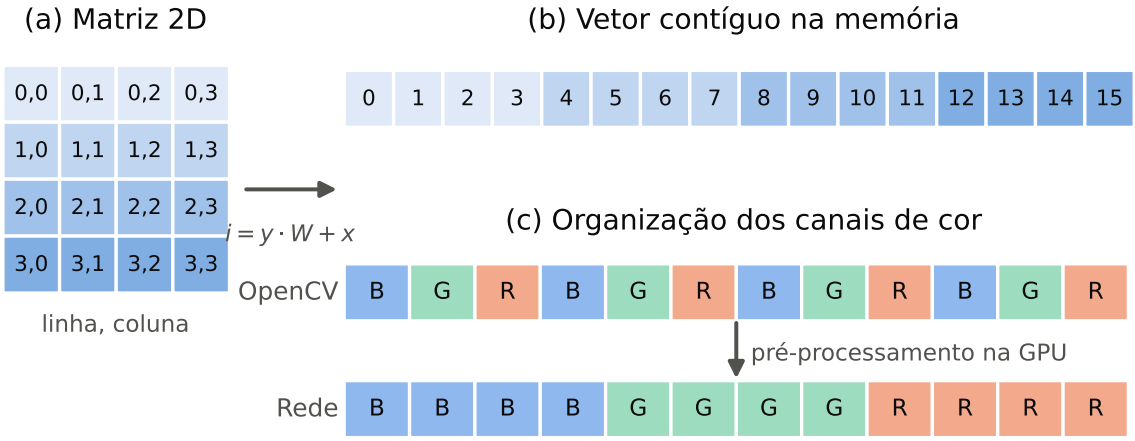
$$index = (y \times W) + x$$

Essa organização favorece a localidade espacial de memória e reduz a ocorrência de acessos não contíguos, característica particularmente importante em aplicações de

processamento paralelo executadas em GPUs (SZELISKI, 2022; KIRK; HWU, 2023). Em algoritmos de interpolação de quadros, nos quais milhões de operações são realizadas para cada sequência de vídeo processada, a forma de armazenamento dos pixels pode influenciar diretamente a latência de execução e o consumo de memória.

A Figura 5 mostra essa organização e a diferença entre as duas formas de dispor os canais de cor utilizadas neste trabalho: a biblioteca OpenCV entrega os canais intercalados, enquanto a rede neural os espera em planos separados.

Figura 5 – Organização da imagem na memória: (a) matriz; (b) vetor contíguo obtido pela linearização; (c) canais intercalados e em planos separados.

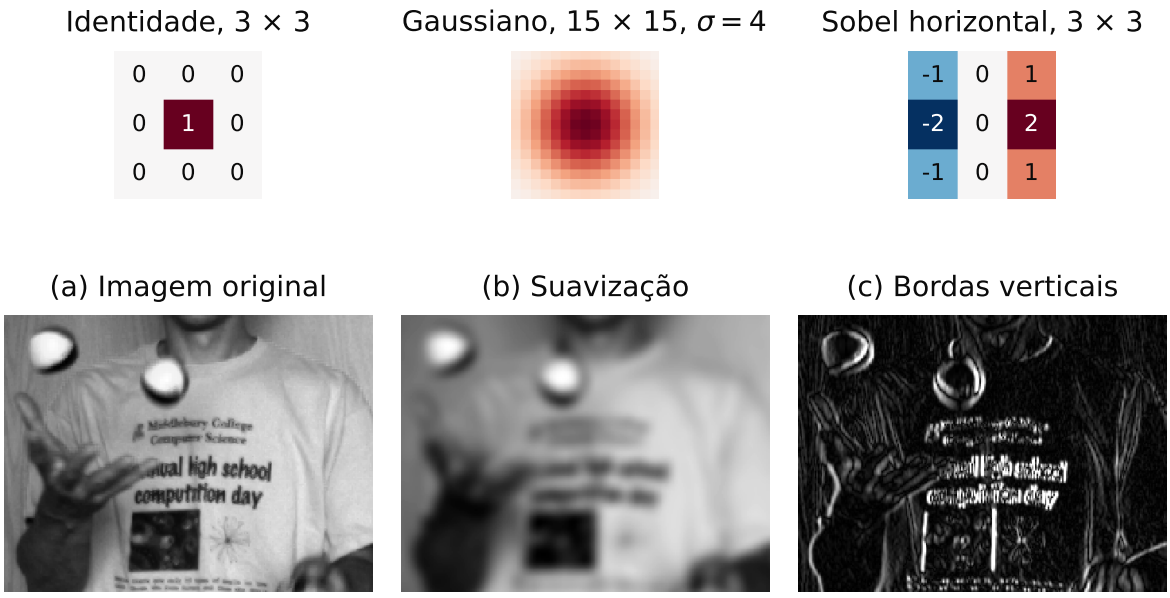


Fonte: Elaborado pelo autor (2026).

2.2.2 Filtragem Espacial e Convolução Adaptativa

A convolução constitui uma das operações fundamentais do processamento digital de imagens e da visão computacional moderna. Em sua forma tradicional, um kernel bidimensional percorre toda a imagem aplicando o mesmo conjunto de coeficientes a cada região analisada. Segundo Gonzalez e Woods (2009), filtros convolucionais clássicos utilizam coeficientes fixos para executar tarefas como suavização, realce de detalhes, remoção de ruído e detecção de bordas. A Figura 6 apresenta dois exemplos: os mesmos coeficientes são aplicados a toda a imagem, independentemente do conteúdo de cada região.

Figura 6 – Filtragem espacial com *kernels* fixos. Na primeira linha, os coeficientes de cada filtro; na segunda, o resultado da sua aplicação.



Fonte: Elaborado pelo autor (2026), a partir da sequência Beanbags do conjunto Middlebury (BAKER et al., 2011).

Matematicamente, a convolução espacial pode ser interpretada como uma operação de combinação ponderada entre os pixels de uma vizinhança local. Em aplicações convencionais de processamento de imagens, a utilização de um kernel fixo é suficiente, uma vez que a transformação desejada é aplicada uniformemente sobre toda a cena. Entretanto, essa abordagem torna-se limitada em problemas de Interpolação de Quadros de Vídeo (VFI), nos quais diferentes regiões da imagem podem apresentar movimentos distintos entre quadros consecutivos (SZELISKI, 2022).

Em cenários de VFI, o objetivo não consiste apenas em filtrar uma imagem existente, mas em sintetizar novas informações visuais correspondentes a um instante temporal intermediário. Dessa forma, um único kernel fixo não é capaz de representar adequadamente a variedade de movimentos presentes na cena. Objetos distintos podem deslocar-se em velocidades e direções diferentes, exigindo estratégias adaptativas para estimar corretamente a posição dos pixels no quadro intermediário.

Para superar essa limitação, surgem as convoluções adaptativas, nas quais os coeficientes do kernel deixam de ser constantes e passam a ser estimados individualmente para cada pixel da imagem. Conforme proposto por Niklaus, Mai e Liu (2017), os coeficientes desses kernels funcionam como indicadores de amostragem, determinando de quais posições espaciais nos quadros de entrada as informações devem ser extraídas para sintetizar o pixel correspondente ao instante intermediário $(t + 0{,}5)$.

Essa estratégia permite incorporar implicitamente informações de movimento durante

o próprio processo de síntese da imagem. Em vez de realizar uma etapa explícita de estimativa de fluxo óptico seguida de reconstrução do quadro, a convolução adaptativa unifica ambas as tarefas em uma única operação. Como resultado, o modelo passa a aprender simultaneamente onde buscar informações e como combiná-las, reduzindo a complexidade do pipeline e possibilitando a geração de quadros intermediários com elevada qualidade visual (NIKLAUS; MAI; WANG, 2021).

A Figura 7 mostra os filtros efetivamente estimados pelo modelo utilizado neste trabalho para dois pixels do quadro intermediário. Em uma região parada, os pesos concentram-se no centro do filtro: o pixel é obtido da mesma posição nos dois quadros. Na bola em movimento, os pesos deslocam-se: o maior peso está 16 pixels abaixo e 4 à esquerda no quadro t, e 14 pixels acima no quadro $t+1$. O filtro indica, portanto, onde buscar a informação em cada quadro, sem que nenhum vetor de movimento tenha sido calculado. Recalculando os dois pixels a partir desses filtros, obtêm-se os valores de saída da implementação original, a menos do truncamento para 8 bits.

Figura 7 – Filtros adaptativos estimados pela rede para dois pixels do quadro intermediário, com os pesos normalizados. O centro de cada filtro (linhas pontilhadas) corresponde à posição do próprio pixel; vermelho indica pesos positivos, e azul, negativos.



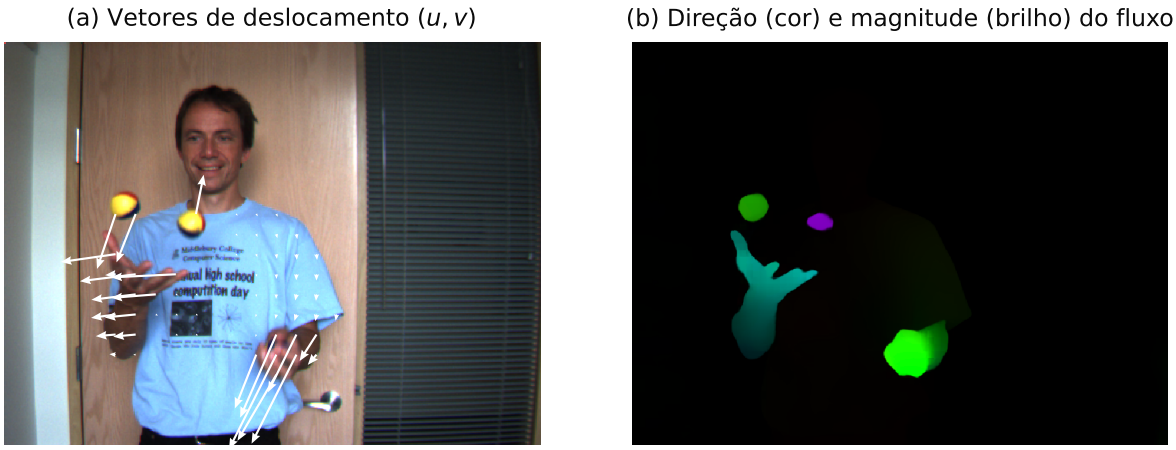
Fonte: Elaborado pelo autor (2026), a partir da sequência Beanbags do conjunto Middlebury (BAKER et al., 2011).

2.2.3 Fluxo Óptico Explícito vs. Kernels de Movimento Implícito

Grande parte dos métodos modernos de Interpolação de Quadros de Vídeo (VFI) baseia-se na estimativa de movimento entre quadros consecutivos. Nesse contexto, o conceito de fluxo óptico desempenha papel fundamental ao descrever o deslocamento aparente dos pixels entre duas imagens ao longo do tempo. Formalmente, o fluxo óptico pode ser representado por um campo vetorial denso, no qual cada pixel possui um vetor (u, v) indicando sua direção e magnitude de deslocamento (SZELISKI, 2022).

Entre os modelos contemporâneos mais relevantes para estimativa de fluxo óptico destaca-se o RAFT (*Recurrent All-Pairs Field Transforms*), proposto por Teed e Deng (2020). O modelo constrói volumes de correlação entre todos os pares de pixels das imagens de entrada e utiliza um mecanismo recorrente para refinar iterativamente os vetores de movimento. Essa estratégia permite atingir elevados níveis de precisão, tornando o RAFT uma das principais referências da área de visão computacional. A Figura 8 mostra o fluxo estimado por esse modelo para a sequência utilizada nas figuras anteriores.

Figura 8 – Fluxo óptico estimado pelo RAFT entre os quadros t e $t+1$: (a) vetores de deslocamento acima de um pixel; (b) direção e magnitude em cada pixel.



Fonte: Elaborado pelo autor (2026), com a implementação do RAFT da biblioteca *torchvision*, a partir da sequência Beanbags do conjunto Middlebury (BAKER et al., 2011). Vetores em (a) ampliados 2,5 vezes.

Após a estimativa do fluxo óptico, métodos tradicionais de VFI realizam uma etapa adicional denominada *warping*, responsável por deformar ou reposicionar os pixels dos quadros originais de acordo com os vetores calculados. Embora eficaz, essa abordagem pode apresentar dificuldades em regiões de oclusão, desoclusão ou movimentos complexos, nas quais determinados pixels tornam-se visíveis ou invisíveis entre quadros consecutivos (TEED; DENG, 2020; SZELISKI, 2022).

Uma alternativa a esse paradigma consiste na utilização de kernels de movimento implícito, abordagem adotada pelos métodos baseados em convolução adaptativa. Nesse

cenário, o movimento não é representado explicitamente por vetores de deslocamento. Em vez disso, a rede neural aprende diretamente um conjunto de coeficientes responsáveis por determinar como os pixels dos quadros de entrada devem ser combinados para sintetizar o valor correspondente ao quadro intermediário (NIKLAUS; MAI; LIU, 2017).

Sob essa perspectiva, os pesos do kernel passam a codificar implicitamente informações de movimento e correspondência espacial. Em vez de calcular uma "seta"indicando o deslocamento de cada pixel, o modelo aprende quais amostras locais devem ser utilizadas e qual contribuição cada uma delas terá na reconstrução do novo quadro. Dessa forma, a estimativa de movimento e a síntese da imagem tornam-se partes de uma única operação computacional. Na sequência das figuras anteriores, as duas abordagens chegam a estimativas próximas do mesmo movimento: o RAFT mede um deslocamento de cerca de $(+7, -35)$ pixels para a bola central, enquanto os filtros da Figura 7 apontam para um deslocamento total de cerca de $(+4, -30)$ pixels entre os dois quadros, sem uma etapa explícita de estimativa de fluxo.

Essa unificação simplifica o *pipeline* de processamento e elimina a necessidade de etapas explícitas de fluxo óptico e *warping*. Além disso, reduz a propagação de erros entre estágios sucessivos do algoritmo, característica que contribuiu para a popularização das abordagens baseadas em convolução adaptativa na literatura recente de interpolação de quadros de vídeo (NIKLAUS; MAI; LIU, 2017; NIKLAUS; MAI; WANG, 2021).

2.2.4 Eficiência via Convoluções Separáveis

O principal desafio das abordagens baseadas em convolução adaptativa está relacionado ao tamanho dos kernels necessários para representar movimentos de grande magnitude. Em métodos de interpolação de quadros, cada pixel do quadro intermediário pode depender de uma ampla região dos quadros de entrada, exigindo filtros com dimensões elevadas. Consequentemente, o custo computacional cresce rapidamente à medida que o tamanho do kernel aumenta (NIKLAUS; MAI; LIU, 2017).

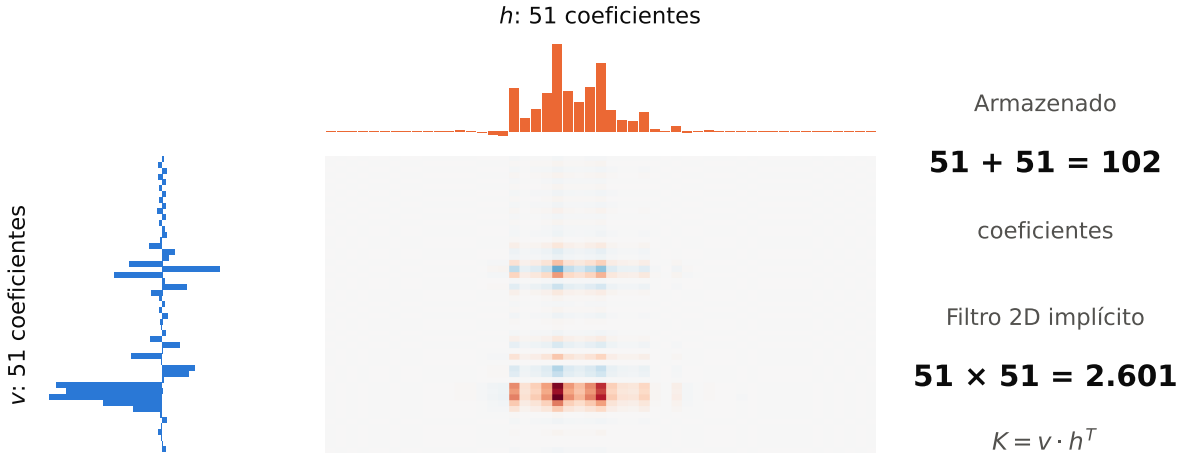
Em uma convolução bidimensional convencional, um kernel de dimensão $N \times N$ contém N^2 coeficientes. Isso significa que cada pixel processado requer a combinação de N^2 amostras da imagem de entrada. Para um kernel de 51×51, por exemplo, são necessárias 2.601 operações de combinação para cada pixel sintetizado. Considerando imagens em resolução HD ou Full HD, esse custo torna-se extremamente elevado tanto em processamento quanto em consumo de memória (NIKLAUS; MAI; LIU, 2017; NIKLAUS; MAI; WANG, 2021).

A técnica de convolução separável explora uma propriedade matemática específica de determinados filtros. Em vez de representar o kernel como uma matriz bidimensional completa, ele é decomposto em dois filtros unidimensionais independentes: um filtro vertical e um filtro horizontal. Matematicamente, essa decomposição pode ser expressa como:

$$K = v \cdot h^T$$

onde v representa o vetor vertical e h representa o vetor horizontal (NIKLAUS; MAI; LIU, 2017). A Figura 9 ilustra a decomposição com os vetores reais estimados pela rede para o pixel da bola da Figura 7.

Figura 9 – Separabilidade: os vetores v e h estimados pela rede para um pixel e o filtro bidimensional implícito formado pelo seu produto. Apenas os dois vetores são armazenados.



Fonte: Elaborado pelo autor (2026), a partir da sequência Beanbags do conjunto Middlebury (BAKER et al., 2011).

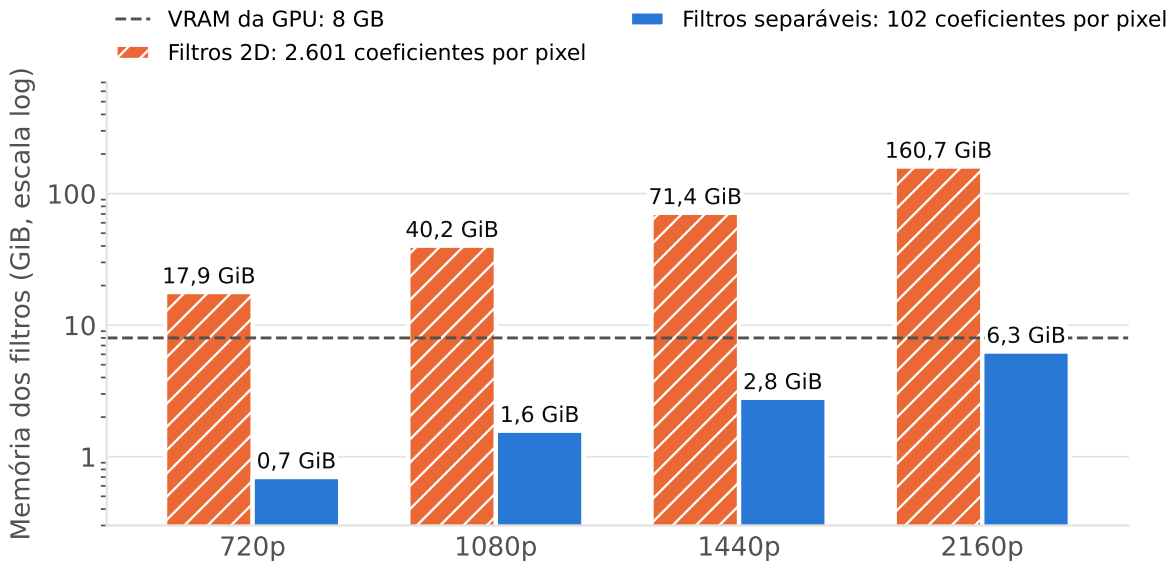
Quando o mesmo filtro é aplicado a toda a imagem, como na filtragem clássica, a separabilidade permite executar a convolução bidimensional em duas passadas unidimensionais sucessivas, compartilhadas entre pixels vizinhos: o custo passa de aproximadamente N^2 para $2N$ operações por pixel. Nas convoluções adaptativas, porém, cada pixel possui o seu próprio par de filtros, e não há passada intermediária a compartilhar; a síntese de cada pixel continua combinando as N^2 amostras da vizinhança (Seção 3.3.2). Nesse caso, o ganho da separabilidade está na quantidade de coeficientes que a rede precisa estimar e armazenar: $2N$ por pixel e por quadro, em vez de N^2. Para $N = 51$, isso representa uma redução de 2.601 para 102 coeficientes por pixel.

Essa diferença possui impacto direto na viabilidade computacional do algoritmo. A redução do número de coeficientes diminui o tamanho da camada de saída da rede, o volume de dados intermediários armazenados na memória durante a inferência e a quantidade de dados lidos pelo *kernel* de síntese. Como resultado, convoluções separáveis tornam possível utilizar kernels de grande alcance espacial sem que a memória necessária se torne proibitiva (NIKLAUS; MAI; LIU, 2017; KIRK; HWU, 2023).

No contexto deste trabalho, essa propriedade é particularmente relevante, pois determina o tamanho dos tensores de filtros mantidos na VRAM durante a inferência. Em

resolução Full HD, os filtros separáveis ocupam cerca de 1,6 GB, enquanto filtros bidimensionais completos exigiriam cerca de 40 GB, valor acima da capacidade de qualquer GPU de consumo. A Figura 10 apresenta esse cálculo para as resoluções avaliadas: com filtros bidimensionais, nem mesmo 720p caberia em 8 GB.

Figura 10 – Memória ocupada apenas pelos tensores de filtros, em FP32, para $N = 51$.



Fonte: Elaborado pelo autor (2026).

2.2.5 Implementação de Baixo Nível e I/O de Vídeo

O desenvolvimento de aplicações de processamento de vídeo em alto desempenho exige atenção não apenas aos algoritmos empregados, mas também à forma como os dados são organizados e transferidos entre os diferentes níveis da arquitetura computacional. Nesse contexto, a linguagem C++ é amplamente utilizada em aplicações de visão computacional e computação de alto desempenho por oferecer controle explícito sobre a alocação de memória, estruturas de dados e gerenciamento de recursos do sistema (KIRK; HWU, 2023).

Em aplicações aceleradas por GPU, os dados normalmente são armazenados inicialmente na memória principal do sistema (Host) e posteriormente transferidos para a memória de vídeo da placa gráfica (Device). Essas transferências ocorrem através do barramento PCI Express (PCIe), cuja largura de banda é significativamente inferior à largura de banda interna da memória da GPU. Consequentemente, movimentações excessivas de dados podem se tornar um gargalo relevante para o desempenho da aplicação (NVIDIA Corporation, 2025).

Para minimizar esse problema, é comum organizar os dados em regiões contíguas de memória antes da transferência para o dispositivo gráfico. Essa estratégia melhora a

localidade espacial dos acessos, reduz a fragmentação dos dados e favorece operações de leitura e escrita mais eficientes tanto na CPU quanto na GPU (KIRK; HWU, 2023).

No motor desenvolvido neste trabalho, os quadros de vídeo são carregados pela biblioteca OpenCV, transferidos para a GPU em regiões contíguas de memória e reorganizados na própria GPU no formato esperado pela rede (Seção 3.3). Essa etapa é particularmente importante em aplicações de interpolação de quadros, uma vez que múltiplas imagens de alta resolução precisam ser mantidas simultaneamente em memória durante o processo de inferência.

2.2.6 Inferência Desacoplada via ONNX Runtime

O Open Neural Network Exchange (ONNX) consiste em um formato intermediário padronizado para representação de modelos de aprendizado profundo, permitindo interoperabilidade entre diferentes frameworks de treinamento e ambientes de inferência (BAI et al., 2019). Nesse ecossistema, o ONNX Runtime atua como um mecanismo otimizado de execução, possibilitando a inferência de redes neurais sem dependência direta do framework originalmente utilizado durante o treinamento (Linux Foundation AI; Microsoft, 2026).

No contexto desta pesquisa, a utilização do ONNX Runtime possui relevância arquitetural, pois permite desacoplar a inferência do ecossistema Python/PyTorch. Além disso, a biblioteca oferece integração nativa com provedores de execução CUDA, permitindo que os tensores permaneçam alocados diretamente na VRAM da GPU durante o pipeline de processamento. Se esse desacoplamento reduz de fato a latência e o consumo de memória é uma das questões verificadas experimentalmente neste trabalho.

Essa abordagem é particularmente importante para a hipótese investigada, uma vez que a redução de transferências desnecessárias entre memória principal e memória de vídeo constitui um fator relevante para mitigação do gargalo conhecido como *Memory Wall* (Seção 2.2.8).

2.2.7 Arquitetura de Decomposição e Garantia de Separabilidade

A viabilidade técnica de utilizar kernels de grandes dimensões (como 51×51) depende da garantia matemática de separabilidade. Para que uma matriz K seja separável, ela deve obrigatoriamente possuir Posto 1 (*Rank-1*), permitindo sua decomposição no produto externo de dois vetores unidimensionais: $K = v \cdot h^T$.

O modelo SepConv resolve esse desafio diretamente em sua arquitetura de rede neural através de uma **bifurcação na camada de saída**. Em vez de estimar os N^2 elementos de uma matriz bidimensional, a rede possui ramificações independentes que inferem, simultaneamente, um vetor vertical v $(N \times 1)$ e um vetor horizontal h $(1 \times N)$ para cada quadro de entrada.

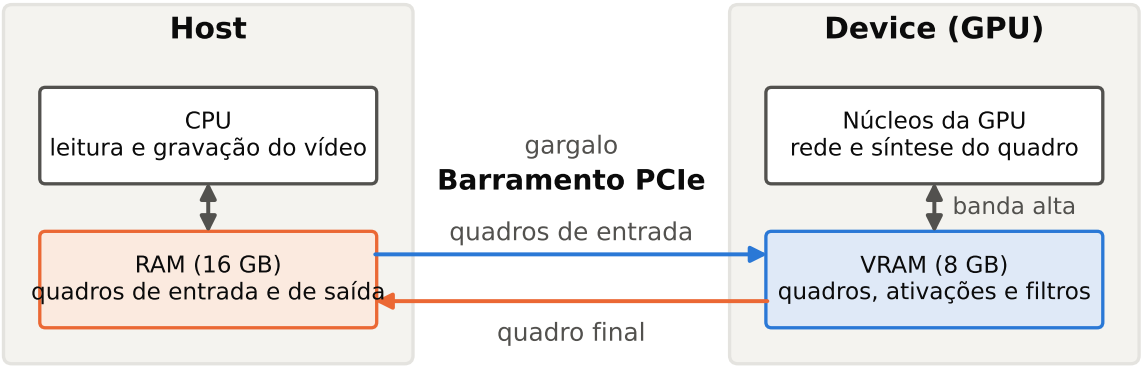
Ao forçar essa estrutura, o algoritmo garante por construção que o kernel resultante tenha posto 1, sem a necessidade de uma decomposição analítica *a posteriori* — que, aplicada a um kernel bidimensional arbitrário, seria apenas uma aproximação. Para a implementação em C++, essa característica é fundamental: o sistema recebe da rede neural apenas $2N$ valores por pixel e por quadro, o que reduz a ocupação de memória pelos filtros. A equivalência numérica entre a síntese realizada pelo motor C++ e a da implementação original é verificada experimentalmente na Seção 3.3.4.

2.2.8 Arquitetura de GPU e o Gargalo de Memória (*Memory Wall*)

A execução de modelos profundos de visão computacional em hardware de consumo é limitada fundamentalmente pelo tráfego de dados e pela largura de banda de memória. Segundo a teoria arquitetural de Hennessy e Patterson (2022), os sistemas computacionais modernos enfrentam o fenômeno denominado *Memory Wall* (Muro da Memória), caracterizado pela disparidade entre o crescimento da capacidade de processamento aritmético e o aumento da largura de banda dos barramentos de memória. Como consequência, em aplicações intensivas em dados, o tempo despendido na movimentação de tensores entre a memória principal e as unidades lógicas frequentemente supera o tempo de execução das operações matemáticas (HENNESSY; PATTERSON, 2022).

Em Unidades de Processamento Gráfico (GPUs), essa restrição exige estratégias rigorosas de alocação e acesso. Conforme aponta Kirk e Hwu (2023), a vazão máxima de processamento de uma GPU depende de acessos à memória global efetuados de forma contígua e coalescida. Transferências frequentes pelo barramento PCI Express (PCIe) ou realocações dinâmicas de estruturas intermediárias na memória de vídeo (VRAM) introduzem contenção de barramento e paralisam os núcleos de processamento. Esse cenário torna-se crítico em GPUs comerciais limitadas a 8 GB de VRAM, que representam a configuração mais difundida entre usuários de computadores pessoais (Valve Corporation, 2026). Nesses ambientes, a redução do *footprint* espacial da inferência — obtida ao estimar $2N$ coeficientes separáveis por pixel em vez de N^2 — constitui um requisito de engenharia essencial para evitar o esgotamento do limite físico de memória. A Figura 11 resume essa organização e o papel do barramento PCIe entre as duas memórias.

Figura 11 – Organização entre Host e Device. O motor desenvolvido transfere pelo barramento PCIe apenas os quadros de entrada e o quadro final.



Fonte: Elaborado pelo autor (2026).

2.2.9 Modelo de Execução Paralela SIMT

Para mitigar os tempos de espera da VRAM e gerenciar milhares de rotinas simultâneas, as arquiteturas de GPU empregam o modelo de execução *Single Instruction, Multiple Threads* (SIMT) (KIRK; HWU, 2023). Diferente do paradigma das CPUs, voltado à redução da latência de instruções sequenciais via memórias *cache* complexas, o modelo SIMT prioriza a vazão total de dados (*throughput*) por meio do paralelismo massivo em nível de dados (KIRK; HWU, 2023; HENNESSY; PATTERSON, 2022).

Sob esse modelo, o hardware organiza as *threads* de execução em grupos lógicos e síncronos denominados *warps* (compostos tipicamente por 32 *threads*) (KIRK; HWU, 2023; NVIDIA Corporation, 2025). O controlador da GPU emite uma única instrução que é executada simultaneamente por todas as *threads* do *warp*, operando sobre diferentes posições de memória ou pixels da imagem (KIRK; HWU, 2023). No contexto do processamento de vídeo e das convoluções adaptativas, esse modelo oferece três vantagens estruturais (GONZALEZ; WOODS, 2009; KIRK; HWU, 2023):

1. **Mapeamento 1:1 de Dados e *Threads*:** A matriz da imagem é particionada diretamente na grade de execução, atribuindo pixels ou vetores unidimensionais de filtros a *threads* específicas, eliminando o *overhead* de laços (*loops*) sequenciais de varredura;

2. **Ocultação de Latência (*Latency Hiding*):** Quando um *warp* é paralisado aguardando uma transação na VRAM, o escalonador do multiprocessador comuta instantaneamente o contexto de execução para outro *warp* cujos dados já estão disponíveis nos registradores, mantendo os núcleos aritméticos continuamente ocupados (KIRK; HWU, 2023);

3. **Coalescência de Memória:** A linearização unidimensional de matrizes de imagem ($index = y \cdot W + x$) permite que *threads* adjacentes de um mesmo *warp* acessem endereços contíguos de VRAM, consolidando múltiplas requisições em uma única transação de barramento (KIRK; HWU, 2023; NVIDIA Corporation, 2025).

A Figura 12 ilustra o mapeamento adotado no motor deste trabalho: uma grade de blocos cobre a imagem, cada bloco contém 32×8 *threads*, e cada *thread* calcula um pixel.

Figura 12 – Mapeamento da imagem na hierarquia de execução da GPU: grade, blocos e *threads*.



Fonte: Elaborado pelo autor (2026), a partir da sequência Beanbags do conjunto Middlebury (BAKER et al., 2011).

2.2.10 Processamento de Baixo Nível e a Plataforma CUDA

A exploração eficiente do modelo SIMT requer interfaces de programação que permitam o controle direto sobre os Multiprocessadores Fluxáveis (*Streaming Multiprocessors* – SMs) da GPU. A plataforma de computação paralela CUDA (*Compute Unified Device Architecture*) fornece extensões à linguagem C++ para a construção de *kernels* — funções executadas paralelamente no dispositivo através de uma hierarquia bidimensional de blocos e *grids* de *threads* (NVIDIA Corporation, 2025).

Um motor de inferência nativo em C++ e CUDA oferece, em princípio, as seguintes possibilidades frente a *frameworks* de alto nível (como PyTorch), que introduzem camadas de abstração e alocações dinâmicas na memória gráfica:

- **Gerenciamento explícito de VRAM:** Permite o controle da alocação de memória e da transferência de dados entre o Host (CPU) e o Device (GPU), com *buffers* alocados uma única vez e reutilizados (NVIDIA Corporation, 2025);
- **Exploração da Memória Compartilhada (*Shared Memory*) e de registradores:** Permite que *threads* armazenem temporariamente os coeficientes dos filtros $1D$ em memórias *on-chip* de baixa latência, reduzindo o tráfego na VRAM global (KIRK; HWU, 2023);
- **Aferição Precisa de Desempenho:** Permite isolar o tempo de cada estágio na GPU por meio de eventos da API (`cudaEventRecord`) (NVIDIA Corporation, 2025).

A integração entre o controle de baixo nível em C++, a redução de coeficientes proporcionada pela separabilidade e a execução paralela em CUDA estabelece a infraestrutura necessária para avaliar experimentalmente os efeitos sobre a latência e o consumo de VRAM em GPUs de consumo.

2.2.11 Métricas de Avaliação Objetiva de Qualidade

A validação de algoritmos de Interpolação de Quadros de Vídeo (VFI) exige a comparação entre o quadro sintetizado (I_{sint}) e o quadro real de referência (I_{real}), também denominado *ground truth*. Para quantificar objetivamente a fidelidade visual das imagens geradas, este trabalho utiliza métricas amplamente adotadas na literatura de processamento de imagens e visão computacional: Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index (SSIM) e Video Multimethod Assessment Fusion (VMAF).

1. Peak Signal-to-Noise Ratio (PSNR):

O PSNR é uma das métricas mais tradicionais para avaliação da qualidade de imagens reconstruídas ou comprimidas. Sua formulação é baseada no Erro Quadrático Médio (*Mean Squared Error* – MSE), que mede a diferença pixel a pixel entre a imagem sintetizada e a imagem de referência (HORE; ZIOU, 2010).

Matematicamente, o PSNR é definido como:

$$PSNR = 10 \cdot \log_{10} \left(\frac{MAX_I^2}{MSE} \right) \tag{2.1}$$

onde MAX_I representa o valor máximo possível de intensidade de um pixel, assumindo normalmente o valor 255 em imagens de 8 bits.

Valores mais elevados de PSNR indicam menor erro de reconstrução e, consequentemente, maior proximidade numérica entre a imagem gerada e a imagem original. Entretanto, embora seja simples e computacionalmente eficiente, essa métrica apresenta limitações perceptuais, pois considera apenas diferenças absolutas entre pixels, sem levar em conta a forma como o sistema visual humano interpreta estruturas, texturas e padrões espaciais (HORE; ZIOU, 2010).

2. Structural Similarity Index (SSIM):

Para contornar as limitações perceptuais do PSNR, Wang et al. (2004) propuseram o *Structural Similarity Index* (SSIM), uma métrica baseada na hipótese de que o sistema visual humano é mais sensível à preservação de estruturas locais do que a diferenças absolutas de intensidade.

O SSIM compara regiões correspondentes entre duas imagens considerando simultaneamente três componentes fundamentais:

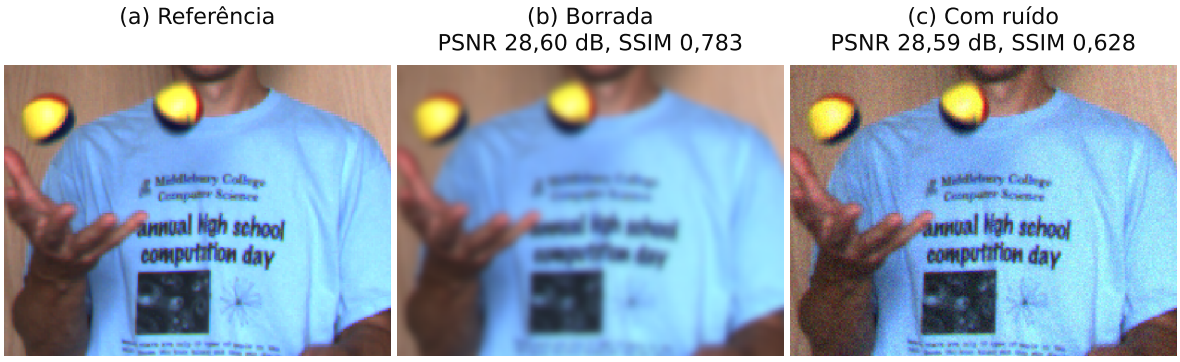
- luminância;
- contraste;

- estrutura.

O resultado é um valor normalmente compreendido no intervalo $[0, 1]$, onde valores próximos de 1 indicam elevada similaridade estrutural entre as imagens comparadas.

Em aplicações de interpolação de quadros, o SSIM é particularmente relevante por ser capaz de identificar degradações visuais frequentemente observadas em quadros sintetizados, como borramento de bordas, perda de detalhes finos e artefatos decorrentes de erros de estimativa de movimento. Por esse motivo, a métrica é amplamente utilizada na literatura contemporânea de VFI como complemento ao PSNR (NIKLAUS; MAI; LIU, 2017; NIKLAUS; MAI; WANG, 2021). A Figura 13 mostra por que as duas métricas se complementam: duas degradações com praticamente o mesmo PSNR, cerca de 28,6 dB, têm SSIM bem diferente (0,783 e 0,628), pois afetam a estrutura da imagem de formas distintas.

Figura 13 – Duas degradações da mesma imagem com PSNR equivalente e SSIM diferente. As métricas são calculadas na imagem inteira; a figura exibe apenas uma região.



Fonte: Elaborado pelo autor (2026), a partir da sequência Beanbags do conjunto Middlebury (BAKER et al., 2011).

3. Video Multimethod Assessment Fusion (VMAF):

Desenvolvido pela Netflix, o *Video Multimethod Assessment Fusion* (VMAF) é uma métrica de qualidade perceptual de vídeo baseada em aprendizado de máquina que combina diferentes algoritmos de avaliação em uma única pontuação (LI et al., 2016). Ao contrário de métricas puramente matemáticas como PSNR e SSIM, o VMAF foi projetado especificamente para refletir a percepção humana da qualidade de vídeo em fluxos contínuos.

O modelo funde três indicadores primários de qualidade perceptual:

- **VIF (*Visual Information Fidelity*):** mede a perda de informação de fidelidade visual em múltiplas escalas espaciais;

- **DLM (*Detail Loss Metric*):** mede a perda de detalhes e a presença de artefatos que distraem a atenção do espectador;
- **Motion:** mede o grau de movimento entre quadros consecutivos para ponderar o impacto do borrão temporal.

A saída do VMAF é mapeada em uma escala de 0 a 100, onde valores próximos a 100 indicam qualidade perceptual idêntica à do vídeo de referência. Em algoritmos de VFI, a inclusão do VMAF justifica-se por sua capacidade de avaliar a consistência temporal da sequência de quadros sintetizada, identificando artefatos dinâmicos (como trepidações e descontinuidades) que métricas quadro a quadro nem sempre capturam com precisão.

A utilização conjunta de PSNR, SSIM e VMAF permite avaliar a precisão numérica, a qualidade estrutural e a fidelidade perceptual da sequência gerada. Dessa forma, torna-se possível analisar os impactos das otimizações propostas neste trabalho não apenas sobre a latência e o consumo de memória, mas também sobre a fidelidade visual e a fluidez temporal dos quadros sintetizados.

2.2.12 Mensuração de Desempenho Computacional

Além da qualidade visual dos quadros sintetizados, a avaliação de algoritmos de Interpolação de Quadros de Vídeo (VFI) envolve aspectos relacionados à eficiência computacional. Em aplicações práticas, especialmente em sistemas de tempo real, a qualidade da imagem deve ser analisada em conjunto com o custo necessário para produzi-la, uma vez que algoritmos visualmente precisos podem apresentar inviabilidade operacional devido ao elevado consumo de recursos computacionais (KIRK; HWU, 2023).

Nesse contexto, duas métricas são frequentemente empregadas na literatura para caracterizar o desempenho de sistemas acelerados por GPU: a latência de processamento e o consumo de memória. A latência representa o tempo necessário para que uma determinada operação seja concluída, influenciando diretamente a capacidade do sistema de processar sequências de vídeo em tempo real. Em aplicações de visão computacional, reduções de poucos milissegundos por quadro podem resultar em aumentos significativos na taxa efetiva de processamento (*throughput*) (HENNESSY; PATTERSON, 2022).

Por sua vez, o consumo de memória constitui um fator limitante para a execução de modelos de aprendizado profundo em hardware de consumo. A necessidade de armazenar simultaneamente quadros de entrada, ativações intermediárias e parâmetros do modelo pode levar ao esgotamento da memória disponível, inviabilizando a inferência mesmo quando a capacidade de processamento da GPU permanece ociosa. Por esse motivo, a utilização eficiente da VRAM tornou-se um aspecto relevante no desenvolvimento de aplicações modernas de inteligência artificial (NVIDIA Corporation, 2025).

No contexto da interpolação de quadros baseada em convoluções adaptativas separáveis, a redução da complexidade espacial representa um dos principais fatores que podem contribuir para a diminuição do consumo de memória e para a melhoria da eficiência geral do sistema. Dessa forma, a análise conjunta de métricas de qualidade visual e desempenho computacional permite uma avaliação mais abrangente dos benefícios e limitações da abordagem investigada neste trabalho.

2.3 Trabalhos relacionados

Nesta seção, são apresentados três trabalhos recentes que abordam o desafio da Interpolação de Quadros de Vídeo (VFI). A seleção prioriza métodos que, assim como esta pesquisa, buscam o equilíbrio entre a fidelidade visual e a eficiência no processamento de grandes deslocamentos de pixels.

2.3.1 Trabalho 1: Revisiting Adaptive Convolutions for Video Frame Interpolation

Niklaus, Mai e Wang (2021) abordaram o problema da complexidade crescente das redes neurais de interpolação, que muitas vezes exigem recursos computacionais excessivos para ganhos marginais de qualidade. Para resolvê-lo, os autores propuseram uma revisitação ao método de convoluções adaptativas, introduzindo melhorias sutis na arquitetura e no treinamento da rede de estimativa de kernels, e denominaram o resultado SepConv++. A metodologia baseou-se no refinamento do modelo SepConv original, otimizando a forma como os filtros $1D$ são gerados. Os resultados mostraram que modelos baseados em kernels podem atingir métricas de PSNR e SSIM próximas ao estado da arte com estruturas mais simples. No entanto, a solução ainda depende de frameworks de alto nível (PyTorch) para execução. O presente trabalho utiliza o modelo pré-treinado disponibilizado por esses autores e diferencia-se por investigar experimentalmente a sua inferência nativa em C++ e CUDA, com foco na latência e na alocação de memória na GPU.

2.3.2 Trabalho 2: Motion-Aware Video Frame Interpolation

Han et al. (2024) abordaram o problema da representação inadequada de movimentos complexos em métodos tradicionais de interpolação de quadros, especialmente em cenários com deslocamentos rápidos, mudanças abruptas de direção e regiões de oclusão. Para enfrentar essa limitação, os autores propuseram uma arquitetura baseada em modelagem de movimento multi-escala, capaz de capturar informações espaciais e temporais em diferentes níveis de resolução.

A metodologia combina a extração hierárquica de características com mecanismos de estimativa de movimento que integram informações provenientes de múltiplas escalas

da imagem. Essa estratégia permite representar movimentos de grande amplitude sem depender exclusivamente de kernels locais ou de campos de fluxo óptico densos, reduzindo artefatos temporais frequentemente observados em sequências complexas.

Os resultados experimentais demonstraram melhorias na preservação da consistência temporal e na qualidade visual dos quadros sintetizados quando comparados a métodos anteriores de interpolação. Em particular, os autores reportam redução de artefatos em regiões com movimentos rápidos e melhor reconstrução de detalhes estruturais.

Entretanto, a arquitetura proposta prioriza a qualidade visual e a robustez da representação de movimento, sem foco específico na redução do consumo de memória ou na otimização da inferência para hardware de consumo. O presente trabalho diferencia-se ao investigar experimentalmente como a separabilidade de kernels e a inferência desacoplada via ONNX Runtime podem contribuir para reduzir a latência e o consumo de VRAM em GPUs comerciais, mantendo níveis aceitáveis de fidelidade visual.

2.3.3 Trabalho 3: RAFT: Recurrent All-Pairs Field Transforms for Optical Flow

Teed e Deng (2020) abordaram a precisão na estimativa de movimento, um problema seminal para a síntese de novos quadros. Para resolvê-lo, propuseram o modelo RAFT, que utiliza uma arquitetura recorrente para atualizar campos de fluxo óptico através de volumes de correlação de todos os pares de pixels. A metodologia utiliza iterações sucessivas para refinar o movimento detectado. Os resultados estabeleceram um novo estado da arte em termos de precisão de vetores. No entanto, a solução apresenta alta latência e exige GPUs de alto desempenho, limitando severamente a viabilidade em cenários comuns. O presente trabalho diferencia-se por adotar a convolução adaptativa como uma alternativa mais contida ao fluxo óptico denso do RAFT, priorizando a performance em GPU e a otimização do consumo de VRAM através de uma inferência nativa.

Tabela 1 – Comparação entre trabalhos relacionados e a abordagem proposta

Autor	**Artigo**	**Estratégia**	**Limitação / Diferencial**
Teed e Deng (2020)	RAFT: Recurrent All-Pairs Field Transforms for Optical Flow	Fluxo óptico recorrente com atualização iterativa	Elevado custo computacional e alto consumo de VRAM.
Niklaus et al. (2021)	Revisiting Adaptive Convolutions for Video Frame Interpolation	Convoluções adaptativas separáveis com refinamentos na estimativa dos *kernels* (SepConv++)	Execução dependente do *framework* PyTorch.
Han et al. (2024)	Motion-Aware Video Frame Interpolation	Modelagem de movimento multi-escala com pirâmide hierárquica e fluxo intermediário	Maior complexidade arquitetural devido ao processamento multiescala.
Este Trabalho	**Inferência SepConv em C++/CUDA**	**Inferência nativa via ONNX Runtime e *kernels* CUDA**	**Análise de latência e uso de VRAM.**

Fonte: Elaborado pelo autor (2026)

3 Desenvolvimento

Neste capítulo, apresenta-se a solução desenvolvida para o problema identificado: o modelo utilizado, a arquitetura do motor de inferência, o processo de exportação e desacoplamento do modelo via ONNX Runtime e a implementação do motor em C++ e CUDA, incluindo a verificação de sua equivalência com a implementação original. O ambiente, o protocolo e os critérios de avaliação são descritos no Capítulo 4.

3.1 Solução Proposta e Arquitetura

A solução proposta para investigar a viabilidade técnica e a eficiência das convoluções adaptativas separáveis consiste no desenvolvimento de um motor de inferência customizado para Interpolação de Quadros de Vídeo (VFI). O foco é transpor a etapa de inferência do modelo SepConv para um ambiente otimizado em GPU.

3.1.1 Modelo utilizado

Os fundamentos da Seção 2.2 descrevem o SepConv original de Niklaus, Mai e Liu (2017). Os experimentos deste trabalho, porém, utilizam a variante revisada **SepConv++** de Niklaus, Mai e Wang (2021), que mantém a formulação de filtros separáveis e introduz melhorias na arquitetura e no treinamento da rede. Foram utilizados o código de referência e os pesos pré-treinados distribuídos pelos autores, com o *checkpoint* `network-paper.pytorch` (a versão avaliada no artigo) e filtros de $N = 51$ coeficientes. O código e os pesos são distribuídos sob a *Adobe Research License*, que permite o uso, a modificação e a redistribuição exclusivamente para fins de pesquisa não comercial, condição atendida por este trabalho. O código original foi mantido sem alterações de comportamento e serve como *baseline* dos experimentos.

3.1.2 Fluxo de dados

O sistema opera em um fluxo de dados (*pipeline*) composto por cinco estágios sequenciais:

1. **I/O (Host):** Leitura e decodificação dos quadros na RAM do sistema via OpenCV (API C++);
2. **Transferência (Host-to-Device):** Envio dos dados linearizados da RAM para a VRAM da GPU via barramento PCIe;

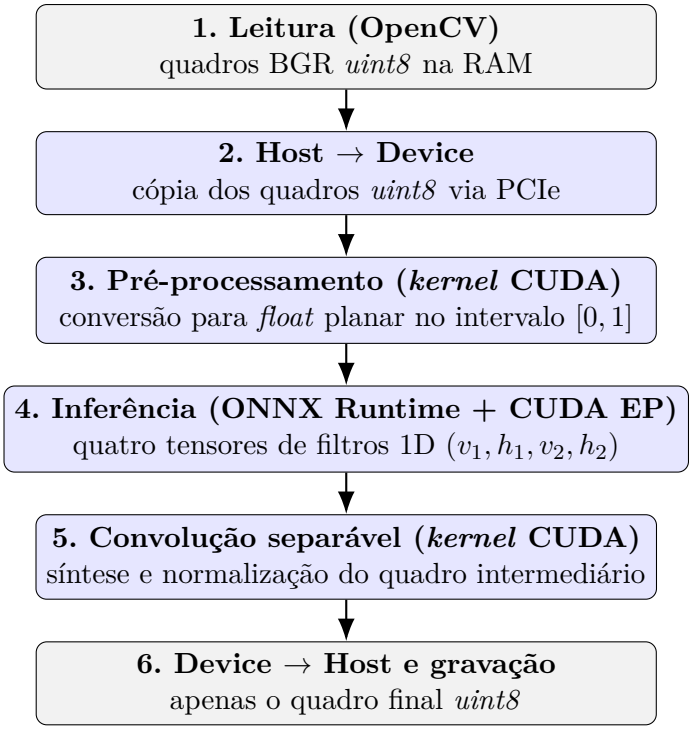
3. **Inferência (ONNX Runtime):** Processamento do modelo pré-treinado para gerar os filtros 1D de cada pixel, mantendo-os alocados na GPU;

4. **Convolução (CUDA):** Síntese paralela do quadro intermediário a partir dos filtros separáveis, por um *kernel* CUDA executado na VRAM;

5. **Retorno e Escrita (Device-to-Host):** Transferência apenas do quadro gerado final de volta para o Host e codificação do vídeo.

A Figura 14 resume o fluxo de dados implementado, destacando quais estágios são executados na memória principal (Host) e quais permanecem inteiramente na memória de vídeo (Device). Na implementação, o pré-processamento dos quadros foi separado em um estágio próprio na GPU.

Figura 14 – Fluxo de dados do motor de inferência SepConv implementado em C++ e CUDA.



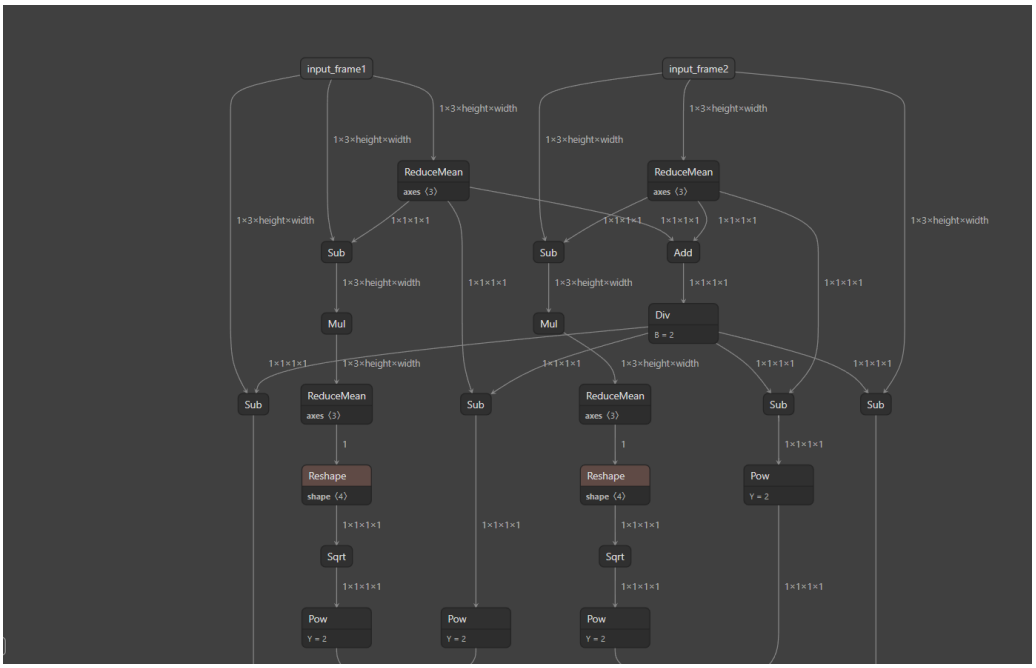
Fonte: Elaborado pelo autor (2026). Em azul, os estágios executados na VRAM.

3.2 Exportação do Modelo e Grafo ONNX

Para desacoplar a inferência do ambiente de treinamento em Python/PyTorch, a rede de estimativa de filtros foi exportada para o formato *Open Neural Network Exchange* (ONNX). O modelo em formato `.onnx` recebe dois quadros de entrada e gera como saída os quatro tensores dos filtros unidimensionais (v_1, h_1, v_2, h_2); a síntese do quadro, que no código original é executada por um *kernel* CUDA à parte, não faz parte do grafo.

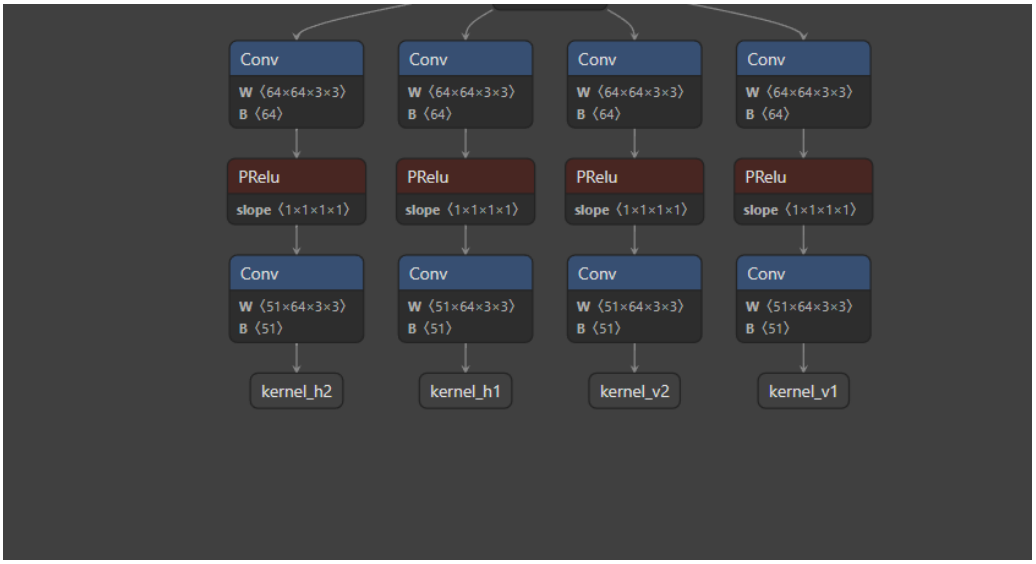
A estrutura do grafo exportado pode ser observada nas Figuras 15 e 16, obtidas a partir da visualização do arquivo `.onnx` gerado.

Figura 15 – Visualização da camada de entrada do grafo computacional SepConv exportado para ONNX.



Fonte: Elaborado pelo autor (2026)

Figura 16 – Bifurcação das camadas de saída no ONNX emitindo os quatro vetores de filtragem $1D$.



Fonte: Elaborado pelo autor (2026)

3.2.1 Correção do grafo para resoluções arbitrárias

A primeira exportação do modelo funcionava apenas na resolução utilizada durante o processo de exportação (1920 × 1080). Em qualquer outra resolução — como 448 × 256 ou 1280 × 720 —, a execução era interrompida por incompatibilidade de dimensões em uma operação de soma do decodificador.

A investigação mostrou que o decodificador original recorta a saída de cada etapa de *upsampling* por meio de uma condição em Python que compara as dimensões dos tensores. Durante a exportação, essa condição é avaliada uma única vez e o recorte resultante é fixado no grafo ONNX, que passa a refletir apenas a sequência de dimensões pares e ímpares da resolução de exportação. A correção consistiu em substituir a condição por um fatiamento (*slicing*) até as dimensões do nível correspondente da rede, operação matematicamente equivalente à original e válida para qualquer resolução de entrada com dimensões pares.

Para verificar a equivalência, as saídas do modelo ONNX (executado pelo ONNX Runtime) foram comparadas às da rede original em PyTorch, com entradas idênticas e o modo TF32 desabilitado nas duas implementações. A Tabela 2 mostra que o erro absoluto máximo permaneceu na ordem de 10^{-6} em termos relativos, compatível com a precisão numérica da aritmética de ponto flutuante de 32 bits.

Tabela 2 – Diferença entre os filtros 1D estimados pelo modelo ONNX e pela rede original

Resolução	**Erro absoluto máximo**	**Maior valor dos filtros**	**Erro relativo**
448 × 256	$5{,}7 \times 10^{-3}$	$3{,}66 \times 10^{3}$	$1{,}6 \times 10^{-6}$
960 × 540	$2{,}0 \times 10^{-3}$	$3{,}66 \times 10^{3}$	$5{,}3 \times 10^{-7}$
1280 × 720	$8{,}2 \times 10^{-3}$	$4{,}60 \times 10^{3}$	$1{,}8 \times 10^{-6}$
1920 × 1080	$9{,}8 \times 10^{-3}$	$4{,}06 \times 10^{3}$	$2{,}4 \times 10^{-6}$
1920 × 1088	$9{,}8 \times 10^{-3}$	$4{,}02 \times 10^{3}$	$2{,}4 \times 10^{-6}$

Fonte: Elaborado pelo autor (2026)

3.3 Implementação do Motor de Inferência

O motor foi implementado como um executável em C++17 que reúne os seis estágios da Figura 14. Todos os *buffers* na VRAM são alocados uma única vez, de acordo com a resolução do vídeo, e reutilizados em todas as interpolações, evitando alocações dinâmicas durante o processamento. O executável opera em dois modos: interpolação de um par de imagens, repetida para fins de medição, e interpolação de vídeos completos com fator 2× ou 4×. No fator 4×, adota-se o mesmo esquema recursivo da implementação original: o quadro $t + 0{,}5$ é sintetizado a partir dos quadros originais e os quadros $t + 0{,}25$ e $t + 0{,}75$, a partir dele e de cada quadro original vizinho. Os experimentos deste trabalho utilizam apenas o fator 2×.

3.3.1 Integração com o ONNX Runtime

O modelo é carregado pela API C++ do ONNX Runtime com o *CUDA Execution Provider*. Para que os tensores não transitem pela memória principal, utiliza-se o mecanismo de *I/O binding*: as duas entradas e as quatro saídas do grafo são associadas diretamente a ponteiros previamente alocados pelo motor na VRAM com `cudaMalloc`. Assim, os quadros pré-processados são lidos pela rede sem cópias adicionais, e os filtros estimados ficam disponíveis na VRAM para o *kernel* de convolução.

3.3.2 *Kernel* de convolução separável

O *kernel* CUDA de síntese atribui uma *thread* a cada pixel (x, y) do quadro intermediário. Sendo I_1 e I_2 os quadros de entrada e v_k, h_k os filtros vertical e horizontal de $N = 51$ coeficientes estimados para o quadro k, cada canal do pixel sintetizado é calculado por:

$$\hat{I}(x,y) = \frac{\displaystyle\sum_{k=1}^{2}\sum_{i=0}^{N-1} v_k(i)\sum_{j=0}^{N-1} h_k(j)\, I_k\Big(c_y(y+i-r),\, c_x(x+j-r)\Big)}{\displaystyle\sum_{k=1}^{2}\Big(\sum_{i} v_k(i)\Big)\Big(\sum_{j} h_k(j)\Big)} \tag{3.1}$$

onde $r = \lfloor N/2 \rfloor = 25$ e c_x, c_y limitam as coordenadas às bordas da imagem. O denominador reproduz a normalização da implementação original, que concatena um canal constante de valor 1 aos quadros e o submete à mesma filtragem; como o *kernel* bidimensional é o produto externo $v\,h^T$, essa soma é obtida diretamente pelo produto das somas dos dois vetores.

Duas decisões de implementação diferem da versão original em PyTorch. Primeiro, a borda replicada de 25 pixels é obtida pela limitação das coordenadas $(c_x,\ c_y)$, sem alocar cópias acolchoadas dos quadros, que no original possuem quatro canais e dimensões $(H + 50) \times (W + 50)$. Segundo, os 51 coeficientes do filtro horizontal são carregados uma única vez em registradores de cada *thread* e reutilizados nas 51 linhas do filtro vertical, e o resultado já é convertido para 8 bits na própria GPU, de modo que apenas o quadro final retorna ao Host.

Cabe uma observação sobre a complexidade discutida na Seção 2.2.4. Como cada pixel possui o seu próprio par de filtros, não é possível decompor a síntese em duas passadas unidimensionais compartilhadas entre pixels vizinhos, como ocorre em uma convolução separável convencional: o laço da Equação 3.1 ainda realiza N^2 multiplicações por pixel. O ganho da separabilidade, neste contexto, está na quantidade de coeficientes que a rede precisa estimar e manter na memória: $2N = 102$ valores por pixel e por quadro, em vez de $N^2 = 2.601$. Em resolução Full HD, os quatro tensores de filtros em ponto flutuante de 32 bits ocupam cerca de 1,6 GB; filtros bidimensionais completos exigiriam aproximadamente 40 GB, inviáveis em qualquer GPU de consumo.

3.3.3 Instrumentação

O motor mede o tempo de cada estágio com eventos CUDA (`cudaEventRecord`) e, para a inferência — executada pelo ONNX Runtime em seu próprio *stream* e de forma síncrona —, com o relógio monotônico do Host. O consumo de VRAM é obtido pela biblioteca NVML, conforme o procedimento descrito na Seção 4.5. Os resultados de cada execução são acrescentados a um arquivo CSV, a partir do qual são gerados os gráficos e tabelas deste trabalho.

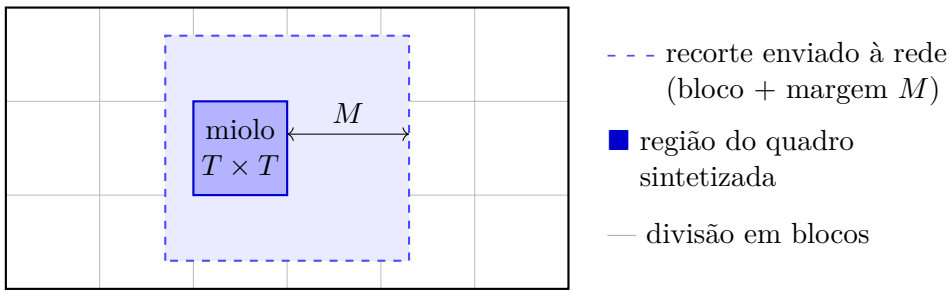
3.3.4 Equivalência com a implementação original

Antes das medições de desempenho, verificou-se se o motor produz o mesmo resultado da implementação original. Para o par de quadros de exemplo distribuído com o código de Niklaus, Mai e Wang (2021) (584 × 388 pixels), o quadro sintetizado pelo motor C++ apresentou PSNR de 93,67 dB e SSIM de 1,000000 em relação ao quadro gerado pela implementação em PyTorch: apenas 0,003% dos valores diferem, e sempre por um único nível de intensidade, efeito do arredondamento de ponto flutuante. Os dois motores são, portanto, equivalentes do ponto de vista da qualidade visual, e as diferenças de desempenho medidas no Capítulo 5 decorrem da forma de execução, e não do resultado calculado.

3.4 Processamento em blocos (segunda versão do motor)

Os resultados da primeira versão (Capítulo 5) mostraram que processar o quadro inteiro de uma só vez esgota a VRAM a partir de 1440p: em 2160p, apenas os quatro tensores de filtros ocupariam cerca de 6,8 GB. Para que o pico de memória deixe de crescer com a resolução, foi implementada uma segunda versão do motor, na qual o quadro é dividido em blocos de $T \times T$ pixels processados um de cada vez. A Figura 17 ilustra a estratégia.

Figura 17 – Processamento em blocos: a rede é executada sobre o bloco acrescido de uma margem, e apenas o miolo do resultado é aproveitado.



Fonte: Elaborado pelo autor (2026)

Três cuidados garantem que o resultado em blocos seja equivalente ao do quadro inteiro:

1. **Normalização global**: a rede normaliza os quadros de entrada pela média e pelo desvio padrão de todos os pixels. Se cada bloco calculasse as próprias estatísticas, blocos com conteúdo diferente seriam normalizados de formas diferentes. Por isso, o modelo foi exportado em uma variante que recebe a média e o desvio como entradas adicionais; o motor os calcula uma única vez por par de quadros, com um *kernel* CUDA de redução, e repassa os mesmos valores a todos os blocos. As saídas dessa variante foram comparadas às da rede original em blocos de diferentes formatos, com erro relativo máximo de 7×10^{-7};

2. **Alinhamento**: a rede reduz a resolução por um fator 16 em seu nível mais profundo. Os blocos e as margens são múltiplos de 16, de modo que cada recorte começa em uma posição múltipla de 16 e a grade interna da rede coincide com a do quadro inteiro;

3. **Margem de sobreposição**: cada pixel sintetizado depende de uma vizinhança dos quadros de entrada, tanto pelo alcance dos filtros (25 pixels) quanto pelo campo receptivo da rede. A rede é executada sobre o bloco acrescido de M pixels de cada lado, e apenas o miolo do resultado é aproveitado.

Em cada bloco, a inferência é seguida imediatamente pela convolução do miolo, que escreve diretamente no quadro de saída. Assim, os tensores de filtros e as ativações da rede passam a ter o tamanho de um bloco com margem, e não o do quadro inteiro; apenas os dois quadros de entrada (cerca de 200 MB em 2160p) e o quadro de saída permanecem inteiros na VRAM. O *kernel* de convolução da Seção 3.3.2 foi generalizado para operar sobre uma região do quadro, lendo os filtros na posição correspondente dentro do recorte. Com $T = 0$, o motor mantém o comportamento da primeira versão.

3.4.1 Escolha da margem

A margem foi definida experimentalmente. Uma reimplementação em PyTorch do processamento em blocos, que reproduz exatamente a saída da implementação original no quadro inteiro, foi executada em três pares de quadros em 1080p para blocos de 256 e 512 pixels e margens de 0 a 160 pixels. A Tabela 3 compara cada configuração com o quadro inteiro.

Tabela 3 – Diferença entre o processamento em blocos e o do quadro inteiro, em 1080p

Bloco	**Margem**	**PSNR (dB)**	**Dif. máx.**	**Pixels dif. (%)**	**Ativações (MiB)**	**ms/quadro**
Quadro inteiro		–	0	0	2.607	383
256	0	53,34	98	5,24	465	431
256	32	64,73	54	0,89	536	497
256	64	77,50	3	0,11	623	653
256	96	95,17	1	0,002	728	820
256	128	111,30	1	<0,0001	845	1.006
256	160	∞	0	0	979	1.251
512	0	57,48	63	2,46	845	382
512	32	66,26	53	0,43	979	461
512	64	81,26	2	0,05	1.120	496
512	96	98,36	1	0,0009	1.242	570
512	128	101,70	1	0,0004	1.421	795
512	160	104,61	1	0,0002	1.509	747

Dif. máx.: maior diferença entre os quadros de 8 bits, em níveis de intensidade. Ativações: pico de memória alocada pelo PyTorch, sem os *buffers* fixos do motor. Fonte: Elaborado pelo autor (2026)

Sem margem, os blocos produzem erros de até 98 níveis de intensidade nas suas bordas. A partir de 96 pixels, a maior diferença em relação ao quadro inteiro passa a ser de um único nível, em menos de 0,002% dos pixels — a mesma ordem do arredondamento observado entre o motor C++ e a implementação original (Seção 3.3.4). Por isso, adotou-se $M = 96$. A tabela também evidencia o compromisso central da estratégia: blocos menores reduzem o pico de memória, mas aumentam o tempo, pois a margem de cada bloco é processada repetidas vezes. Com blocos de 512 pixels e margem de 96, o pico de ativações caiu 52% e o tempo aumentou 49% em relação ao quadro inteiro.

No motor C++, a saída em blocos ($T = 512$ e $T = 256$, $M = 96$) diferiu da saída do quadro inteiro em no máximo um nível de intensidade, em 0,02% dos pixels. A avaliação de desempenho desta segunda versão segue o mesmo protocolo da primeira (Capítulo 4) e é apresentada na Seção 5.4.

3.5 Alocador de memória e meia precisão (terceira versão do motor)

As duas primeiras versões deixaram duas fontes de consumo de memória sem tratamento: a gestão de memória do ONNX Runtime, apontada como causa provável do maior consumo da primeira versão (Seção 5.2), e a precisão numérica, já que todos os tensores eram mantidos em ponto flutuante de 32 bits (FP32). A terceira versão do motor atua sobre as duas sem alterar a estratégia de processamento, e pode ser usada tanto com o quadro inteiro quanto em blocos.

3.5.1 Configuração do alocador do ONNX Runtime

Para avaliar as opções do alocador, a medição de VRAM precisou ser alterada. Até a segunda versão, a NVML era consultada após cada interpolação, o que basta enquanto a memória reservada permanece reservada; quando o ONNX Runtime devolve memória ao fim de cada execução, porém, o pico ocorre durante a execução e não seria registrado. Passou-se então a consultar a NVML a cada milissegundo, em uma *thread* separada, durante toda a execução, tanto no motor quanto na *baseline*. Repetidas com essa medição, as condições das duas primeiras baterias apresentaram picos até 0,5% diferentes dos originais, de modo que os resultados anteriores permanecem válidos.

Cada opção foi executada em 1080p, com o quadro inteiro e com blocos de 512 pixels (Tabela 4). A opção de maior efeito foi o **padrão de memória** (*memory pattern*), mecanismo pelo qual o ONNX Runtime planeja as alocações dos tensores intermediários a partir das primeiras execuções e passa a reservá-los em um único bloco. Desativá-lo reduziu o pico do quadro inteiro de 6.646 MiB para 4.743 MiB (29%), sem alterar o tempo: a memória atribuída ao ONNX Runtime e ao contexto CUDA caiu de cerca de 5,0 GB para 3,1 GB. Uma explicação provável, não verificada, é que a memória das primeiras execuções permanece na *arena* ao lado do bloco planejado. Nos blocos, o maior efeito foi o da estratégia de crescimento da *arena*: em potências de dois, o pico caiu 25%, possivelmente porque blocos de memória maiores são reaproveitados entre recortes de formatos diferentes. Liberar a *arena* após cada execução também reduziu a memória, mas aumentou o tempo em 18% a 27%; as opções relacionadas ao cuDNN e aos pesos da rede não tiveram efeito.

Tabela 4 – Efeito das opções do alocador do ONNX Runtime no motor C++, em 1080p

	Quadro inteiro		**Blocos 512**	
Configuração	**ms**	**MiB**	**ms**	**MiB**
Configuração anterior (com padrão de memória, *arena* sob demanda)	307,3	6.646	650,9	2.170
Arena em potências de dois	305,2	6.929	658,1	1.625
Algoritmos do cuDNN por heurística	306,2	6.643	653,2	2.171
Workspace do cuDNN limitado	306,0	6.643	653,9	2.172
Pesos alocados fora da *arena*	306,0	6.650	650,0	2.171
Sem padrão de memória	305,7	4.743	650,6	1.831
Arena liberada após cada execução	363,2	4.743	828,0	1.634
Sem padrão de memória, *arena* em potências de dois	306,2	4.885	653,0	1.631
Todas as opções, exceto potências de dois	341,0	4.743	742,2	1.291

Média de sete medições por configuração. Em negrito, a configuração adotada na terceira versão. Fonte: Elaborado pelo autor (2026)

Adotou-se a combinação sem padrão de memória e com a *arena* em potências de dois, a de menor memória sem custo de tempo nos blocos; no quadro inteiro, ela consome 3% a mais que a desativação isolada do padrão de memória. Nas resoluções mais altas, o efeito foi maior: com blocos de 1024 pixels, o pico caiu de 5.805 MiB para 3.501 MiB em 1440p

e, em 2160p, de 7.247 MiB, com a VRAM saturada, para 4.655 MiB, sem saturação. O quadro inteiro em 1440p continuou saturando a VRAM em todas as configurações, pois apenas os tensores de filtros ocupam cerca de 3 GB. Em todos os casos, a saída diferiu da configuração anterior em no máximo um nível de intensidade, a mesma diferença observada entre duas execuções idênticas, decorrente da escolha de algoritmos do cuDNN.

3.5.2 Inferência em meia precisão (FP16)

Os dois modelos ONNX (quadro inteiro e blocos) foram convertidos para meia precisão (FP16) com a biblioteca *onnxconverter-common*, o que reduz o arquivo de cada modelo de 54 MB para 26 MB. Os pesos e os tensores intermediários passam a FP16, mas as entradas permanecem em FP32, de modo que o pré-processamento do motor não muda. As saídas — os quatro tensores de filtros — também passam a FP16, o que reduz à metade a memória dos filtros (de 1,6 GB para 0,8 GB em 1080p), e não apenas a das ativações. No modelo do quadro inteiro, as operações que calculam a média e o desvio padrão dos quadros foram mantidas em FP32, porque a soma de milhões de valores em FP16 perderia precisão.

O *kernel* de convolução separável (Seção 3.3.2) passou a aceitar filtros em FP16: cada coeficiente é convertido para FP32 na leitura, e a acumulação continua em FP32. O motor identifica a precisão pelo tipo da saída do modelo. Em três pares de quadros em 1080p, os filtros em FP16 diferiram dos em FP32 em no máximo 1,5% do maior valor, e o quadro sintetizado diferiu em no máximo um nível de intensidade, em cerca de 0,13% dos pixels — diferença da mesma ordem do arredondamento já observado entre o motor e a implementação original (Seção 3.3.4). A avaliação de desempenho desta versão é apresentada na Seção 5.5.

4 Metodologia

Este capítulo descreve o delineamento dos experimentos: as condições comparadas, o ambiente, o conjunto de avaliação, o procedimento de medição, as métricas, os critérios de decisão sobre a hipótese e os riscos identificados. O mesmo protocolo é aplicado a todas as condições, de modo que as diferenças observadas possam ser atribuídas à forma de execução e não a diferenças de metodologia.

4.1 Delineamento e condições experimentais

Trata-se de um estudo experimental comparativo. As variáveis independentes são a **implementação** e a **resolução** do vídeo; as variáveis dependentes são a latência por quadro, a taxa de quadros por segundo, o pico de VRAM e a fidelidade visual (PSNR, SSIM e VMAF). Foram definidas seis condições:

1. ***Baseline* Python/PyTorch**: o código original de Niklaus, Mai e Wang (2021) (Seção 3.1.1), em que a rede é executada pelo PyTorch e a síntese por um *kernel* CUDA compilado em tempo de execução. Um *script* de medição invoca o código original sem alterar o seu comportamento e registra o tempo por quadro e a VRAM com os mesmos critérios do motor C++;

2. **Motor C++/CUDA**: o motor descrito na Seção 3.3, com a rede executada pelo ONNX Runtime e a síntese pelo *kernel* próprio;

3. **Controle ONNX Runtime**: a rede executada pelo ONNX Runtime, mas sem o *kernel* próprio de síntese, para separar o efeito da troca do mecanismo de inferência do efeito do *kernel* de convolução. Esta condição foi definida no protocolo, mas os seus resultados não estão incluídos nesta versão do texto (Seção 4.7);

4. **Motor C++/CUDA em blocos**: a segunda versão do motor (Seção 3.4), com blocos de 256, 512 e 1024 pixels e margem de 96 pixels;

5. **Motor C++/CUDA com o alocador ajustado e em meia precisão**: a terceira versão do motor (Seção 3.5), em FP16 com o quadro inteiro e com blocos de 1024 e 512 pixels, e em FP32 com o quadro inteiro, para separar o efeito do alocador do efeito da precisão;

6. ***Baseline* em meia precisão**: o mesmo código original da condição 1, com a execução envolvida pelo `torch.autocast`, o mecanismo de precisão mista do PyTorch. O código não é alterado: a rede calcula em FP16, e a função de síntese, que já declara suporte a esse mecanismo, converte os filtros para FP32 antes da convolução.

Como as condições 1 e 2 utilizam o mesmo modelo e os mesmos filtros separáveis, a comparação entre elas isola o efeito da forma de execução (linguagem, mecanismo de inferência e gerenciamento de memória), e não o efeito da separabilidade em si, que está presente nas duas. Na condição 5 em FP16, muda também a precisão numérica; a comparação com a condição 1 mede, nesse caso, o efeito conjunto da forma de execução e da precisão, e a condição 6 permite separar os dois efeitos (Seção 5.6).

4.2 Ambiente Experimental

Todos os experimentos foram executados em um computador pessoal com configuração representativa do perfil de hardware de consumo discutido na Seção 1.4. A Tabela 5 descreve o hardware e as versões de software utilizadas, informação necessária para a reprodutibilidade dos resultados.

Tabela 5 – Hardware e software do ambiente experimental

Componente	**Especificação**
GPU	NVIDIA GeForce RTX 4060 Ti, 8 GB de VRAM (arquitetura Ada Lovelace, *compute capability* 8.9)
CPU	Intel Core i5-12400F
Memória principal	16 GB
Sistema operacional	Windows 11 Home (*driver* NVIDIA 610.47 na primeira bateria e 610.88 nas demais)
Motor C++	C++17, CUDA Toolkit 13.3, MSVC (Visual Studio 2022), CMake 4.4
Inferência	ONNX Runtime 1.23.2 (GPU) com *CUDA Execution Provider*; conversão para FP16 com *onnxconverter-common* 1.16
Entrada e saída de vídeo	OpenCV 4.13 (API C++ e Python)
Baseline	Python 3.10, PyTorch 2.11 (CUDA 12.8), implementação original de Niklaus, Mai e Wang (2021)
Métricas	scikit-image 0.25 (PSNR e SSIM), FFmpeg 9.0 com *libvmaf* (VMAF)

Fonte: Elaborado pelo autor (2026)

Os *clocks* da GPU não foram fixados: a placa operou com o gerenciamento de energia padrão do *driver*. As condições da GPU no início da bateria (ocupação de memória, temperatura e *clock*) foram registradas junto com os resultados, e o efeito da variação de *clock* é mitigado pelo aquecimento e pelas medições repetidas descritos na Seção 4.4.

4.3 Conjunto de avaliação

Os testes utilizam um trecho do segundo trailer do jogo GTA VI (Rockstar Games, 2025), disponível em resolução 3840 × 2160 a 30 FPS, escolhido por oferecer conteúdo em resolução nativa 4K com movimentos de câmera e de personagens. O material é protegido por direitos autorais e foi utilizado exclusivamente para avaliação científica, sem redistribuição dos vídeos.

Como o trailer é composto por muitas cenas curtas, foi realizada uma detecção automática de cortes de cena, que identificou sete transições no intervalo de 20 segundos disponível. Cortes de cena não possuem movimento a ser interpolado e reduzem artificialmente as métricas de qualidade — em uma execução preliminar, um único corte produziu um quadro com PSNR de 12,5 dB. Por isso, foi selecionado um trecho de 61 quadros (aproximadamente 2 segundos) contido no maior intervalo sem cortes.

O trecho foi convertido para quatro resoluções, que compõem a grade de resoluções avaliada: 1280 × 720 (720p), 1920 × 1080 (1080p), 2560 × 1440 (1440p) e 3840 × 2160 (2160p). Todos os vídeos de teste e de saída são gravados com o codec sem perdas FFV1, para que a compressão não interfira nas métricas de qualidade.

Para relacionar a fidelidade visual aos conjuntos de referência da literatura, as implementações foram avaliadas também em dois deles. O primeiro são as 12 sequências do Middlebury que possuem o quadro intermediário real, distribuídas com o código de Niklaus, Mai e Wang (2021), com resolução de até 640 × 480 pixels. O segundo é uma amostra de 200 trios de quadros do conjunto de teste do Vimeo-90K (XUE et al., 2019) (448 × 256 pixels), sorteados com semente fixa entre os 3.782 trios do conjunto; em cada trio, o quadro central é sintetizado a partir dos outros dois. Como o servidor oficial do conjunto recusou o *download*, os dados foram obtidos de uma cópia pública no Hugging Face, que não documenta a sua procedência; a cópia tem o mesmo número de trios, a mesma organização de pastas e a mesma resolução do conjunto oficial. Nesses conjuntos, avaliou-se apenas a fidelidade visual, por PSNR e SSIM: as imagens são pequenas demais para as medições de desempenho, e o VMAF é uma métrica de vídeo.

4.4 Procedimento de medição

Para cada resolução, gera-se uma versão do trecho contendo apenas os quadros pares (15 FPS). Cada implementação interpola essa versão com fator 2×, reconstruindo um vídeo de 30 FPS no qual os quadros de índice ímpar são sintetizados. Esse procedimento permite medir o desempenho e a qualidade em uma única execução: os 30 quadros sintetizados são comparados aos quadros reais correspondentes do trecho original, que funcionam como *ground truth*.

As duas implementações leem e gravam os vídeos com a mesma biblioteca (OpenCV) e

o mesmo esquema de interpolação. Cada combinação de implementação e resolução foi executada uma vez, em sequência, por um único *script*. A segunda e a terceira versões do motor foram avaliadas em uma segunda e em uma terceira bateria, cada uma repetindo também a *baseline* e o motor com o quadro inteiro, para que as condições fossem comparadas nas mesmas circunstâncias (Seções 5.4 e 5.5). Uma quarta bateria mediu, nas mesmas circunstâncias, as quatro combinações de implementação e precisão com o quadro inteiro (Seção 5.6). As três primeiras interpolações de cada execução são descartadas como aquecimento — período em que ocorrem a inicialização do contexto CUDA, a seleção de algoritmos do cuDNN e a reserva inicial de memória —, e as demais (27 por resolução) compõem a média e o desvio padrão do tempo por quadro. Em 2160p, devido ao tempo de execução, foram processados apenas cinco quadros de entrada, com uma interpolação de aquecimento, resultando em três medições de tempo e quatro quadros sintetizados avaliados.

4.5 Métricas

- **Latência**: tempo médio para sintetizar um quadro, incluindo a transferência dos quadros para a GPU, a inferência, a convolução e o retorno do resultado ao Host, acompanhado do desvio padrão entre as medições. No motor C++, o tempo também é decomposto por estágio (Seção 3.3.3); na *baseline*, nesta bateria, registrou-se apenas o tempo total;

- **Taxa de quadros (FPS)**: quadros sintetizados por segundo, obtida como o inverso da latência média;

- **Consumo de VRAM**: como o *driver* do Windows não informa à NVML o consumo por processo, registra-se a ocupação total da GPU antes da inicialização da implementação (linha de base) e o pico observado durante a execução; o consumo atribuído à implementação é a diferença entre ambos. Nas duas primeiras baterias, a ocupação foi lida após cada interpolação; a partir da terceira, a cada milissegundo, por uma *thread* separada, o que se mostrou necessário para o estudo do alocador (Seção 3.5) e não alterou os picos das condições anteriores em mais de 0,5%. Para que a linha de base seja estável, nenhuma outra aplicação de uso intenso da GPU é executada durante as medições. No Windows, quando a VRAM se esgota, o *driver* passa a utilizar a memória principal do sistema (*Sysmem Fallback*) em vez de interromper a execução, o que eleva drasticamente a latência. Como a NVML não contabiliza essa memória, as execuções em que a VRAM foi saturada são sinalizadas: no motor C++, quando a ocupação atinge 97% da capacidade; na *baseline*, também quando a memória reservada pelo PyTorch excede a VRAM livre no início da execução;

- **Fidelidade visual**: PSNR, SSIM e VMAF calculados exclusivamente sobre os quadros sintetizados. Incluir os quadros pares, que são cópias dos originais, elevaria artificialmente os resultados. Pelo mesmo motivo, as tarjas pretas horizontais presentes no trailer (*letterbox*, 20% da altura do quadro), reconstruídas sem erro por qualquer método, são excluídas da área avaliada. O VMAF é calculado pelo filtro *libvmaf* do FFmpeg (Netflix, 2026) com o modelo padrão `vmaf_v0.6.1`; como verificação, a comparação do vídeo de referência com ele mesmo resultou em 99,91, valor esperado para sequências idênticas.

4.6 Critérios de decisão e análise estatística

A hipótese é avaliada pelos três critérios da Seção 1.3.3, em cada resolução:

- **H1 (latência)**: as 27 medições de tempo de cada implementação são comparadas pelo teste t de Welch unilateral, que não pressupõe variâncias iguais, ao nível de significância $\alpha = 0{,}05$. O critério é atendido se a diferença for significativa e a redução média for de pelo menos 10%, limiar adotado para que a diferença seja relevante na prática, e não apenas estatisticamente detectável;
- **H2 (VRAM)**: o pico de VRAM é uma medida única por execução, de modo que não se aplica teste estatístico. O critério é atendido se o pico do motor for no máximo 90% do pico da *baseline* e se a execução não tiver saturado a VRAM;
- **H3 (fidelidade)**: as métricas são comparadas quadro a quadro entre as implementações, sobre os mesmos quadros sintetizados. O critério é atendido se a diferença média não exceder 0,1 dB de PSNR, 0,001 de SSIM e 0,5 ponto de VMAF, limites inferiores à variação observada entre quadros de uma mesma execução.

Os limiares numéricos foram fixados durante a revisão do texto, após a primeira bateria de testes. Para não ajustar os critérios aos resultados, os mesmos limiares são mantidos para as baterias seguintes e os resultados são relatados mesmo quando contrários à hipótese.

4.7 Riscos e mitigações

A Tabela 6 resume os riscos identificados para a conclusão do trabalho e as mitigações adotadas.

Tabela 6 – Riscos identificados e mitigações

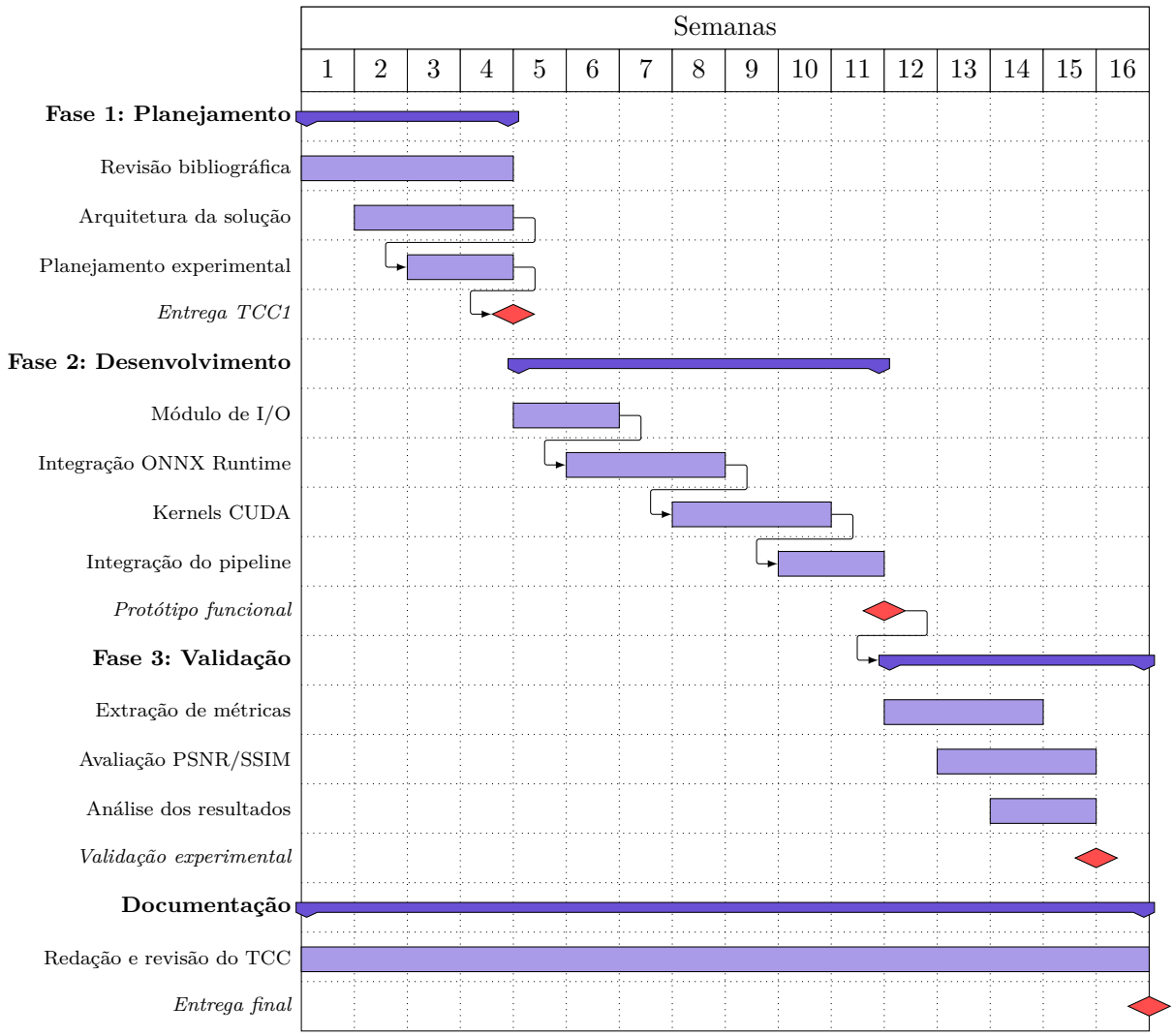
Risco	**Mitigação**
Os ajustes do motor — processamento em blocos, alocador do ONNX Runtime e inferência em FP16 — não ficarem prontos a tempo	Os ajustes foram implementados e avaliados em ordem de prioridade, cada um em uma bateria própria (Seções 5.4 e 5.5)
A VRAM se esgotar em 1440p e 2160p e o *driver* passar a usar a memória principal	Execuções sinalizadas como saturadas e analisadas separadamente; a hipótese é avaliada em 720p e 1080p; o processamento em blocos (Seção 3.4) é avaliado como forma de evitar a saturação
Base de avaliação restrita a um único trecho de vídeo	Avaliação complementar da fidelidade no Middlebury e no Vimeo-90K; delimitação explícita do alcance das conclusões
Variação de *clock* e de ocupação da GPU entre execuções	Aquecimento, 27 medições por condição, registro das condições da GPU e repetição das baterias
Condição de controle (ONNX Runtime sem *kernel* próprio) não executada	Relatada como limitação; sem ela, a diferença de latência é atribuída ao conjunto da forma de execução, e não a um componente específico

Fonte: Elaborado pelo autor (2026)

4.8 Cronograma de trabalho

O cronograma a seguir apresenta, no formato de diagrama de Gantt, as atividades previstas para o desenvolvimento do TCC2, distribuídas ao longo das semanas do semestre.

Figura 18 – Cronograma de atividades – Diagrama de Gantt



Fonte: Elaborado pelo autor (2026)

5 Resultados e Discussão

Este capítulo apresenta os resultados da primeira versão do motor C++/CUDA, que reproduz a estratégia de memória da implementação original: todos os tensores do quadro inteiro são processados de uma só vez. A Tabela 7 consolida as medições, detalhadas nas seções seguintes. Os resultados da segunda versão, com processamento em blocos, e da terceira, com o alocador ajustado e meia precisão, são apresentados nas Seções 5.4 e 5.5; a Seção 5.6 separa o efeito da forma de execução do efeito da precisão, a Seção 5.7 avalia a fidelidade nos conjuntos de referência, e a Seção 5.8 analisa as três versões frente à hipótese.

Tabela 7 – Desempenho e fidelidade visual por resolução (interpolação 2×)

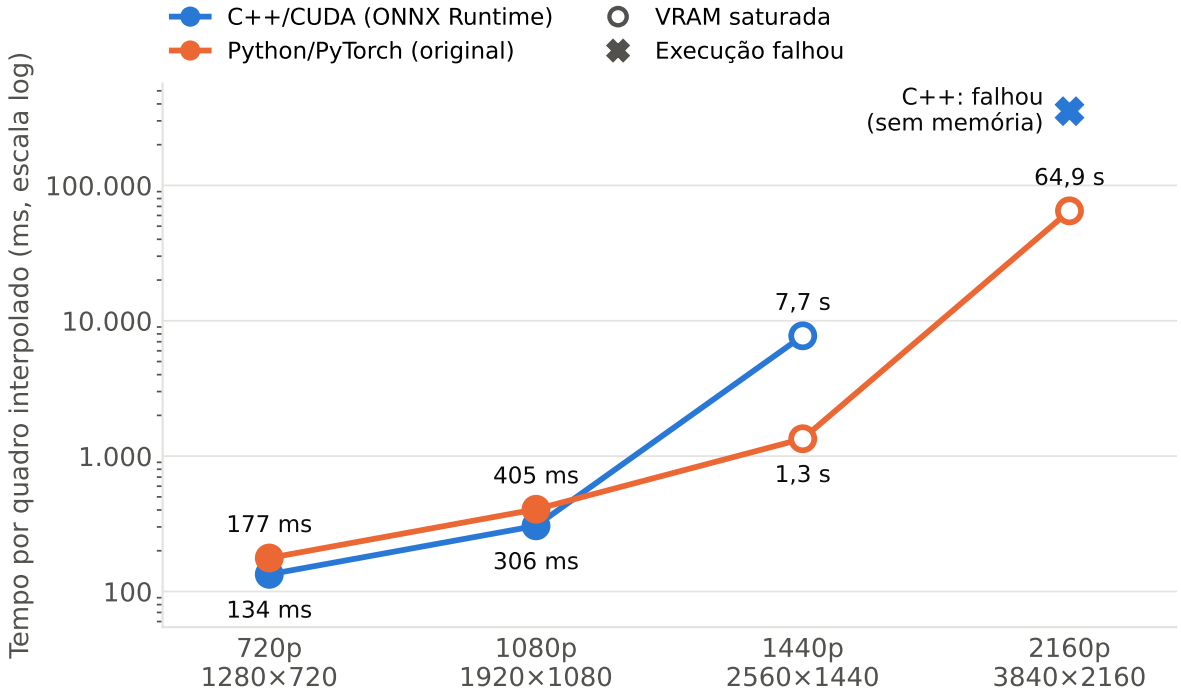
Res.	Implementação	ms/quadro	FPS	VRAM (MiB)	PSNR (dB)	SSIM	VMAF
720p	C++/CUDA	133,9 ± 0,4	7,47	3.138	34,48	0,9622	85,16
720p	Python/PyTorch	176,8 ± 1,0	5,65	2.149	34,48	0,9622	85,16
1080p	C++/CUDA	305,5 ± 1,8	3,27	6.648	34,10	0,9538	77,42
1080p	Python/PyTorch	404,9 ± 2,2	2,47	4.585	34,10	0,9538	77,42
1440p	C++/CUDA	7.745,1 ± 214,5*	0,13	7.264*	34,04	0,9532	71,91
1440p	Python/PyTorch	1.344,5 ± 3,8*	0,74	7.324*	34,04	0,9532	71,91
2160p	C++/CUDA	falhou por falta de memória					
2160p	Python/PyTorch	64.900,5 ± 3.452,8*	0,02	7.133*	31,44	0,9323	50,33

Latência: média ± desvio padrão. * VRAM saturada: o excedente foi alocado na memória principal e não é contabilizado pela NVML. Fonte: Elaborado pelo autor (2026)

5.1 Latência

A Figura 19 apresenta o tempo médio por quadro interpolado. Em 720p e 1080p, resoluções em que a VRAM foi suficiente para as duas implementações, o motor C++ foi cerca de 24% mais rápido que a *baseline* (133,9 ms contra 176,8 ms e 305,5 ms contra 404,9 ms), com desvio padrão inferior a 1% da média. A partir de 1440p, a VRAM se esgota nas duas implementações e o comportamento se inverte: o motor C++ passa a levar 7,7 s por quadro, cerca de 5,8 vezes mais que a *baseline* (1,3 s), e em 2160p não consegue alocar memória para a inferência. A implementação em PyTorch conclui o 2160p, porém com 64,9 s por quadro, pois a maior parte dos tensores passa a residir na memória principal.

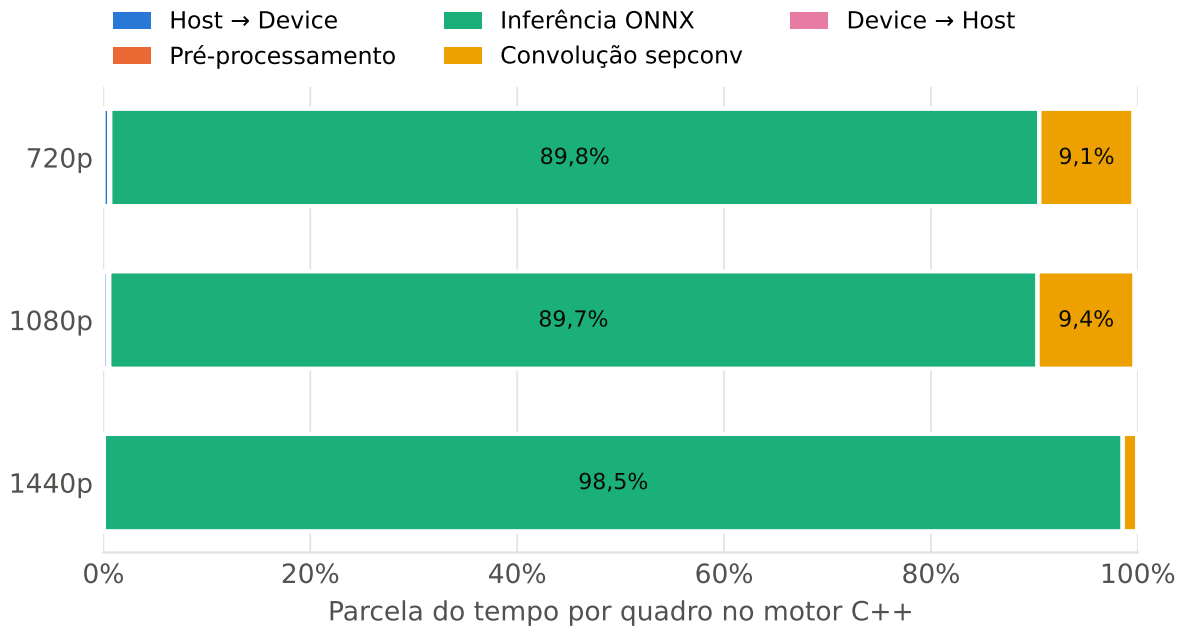
Figura 19 – Tempo médio por quadro interpolado em função da resolução (escala logarítmica).



Fonte: Elaborado pelo autor (2026). Marcadores vazados indicam VRAM saturada.

A Figura 20 mostra como o tempo do motor C++ se distribui entre os estágios do *pipeline*. A inferência da rede pelo ONNX Runtime responde por cerca de 90% do tempo (274,2 ms em 1080p), seguida pela convolução separável (28,7 ms, 9,4%); as transferências entre Host e Device e o pré-processamento somam menos de 1%. Esse resultado indica que, nesta versão, a transferência pelo barramento PCIe não é um gargalo relevante, e que o tempo total depende principalmente da etapa de inferência. Em 1440p, a participação da inferência sobe para 98,5%, consequência direta do uso da memória compartilhada. Como a *baseline* foi medida apenas pelo tempo total, esta bateria não permite atribuir a redução de latência a um estágio específico.

Figura 20 – Distribuição do tempo por quadro entre os estágios do motor C++.

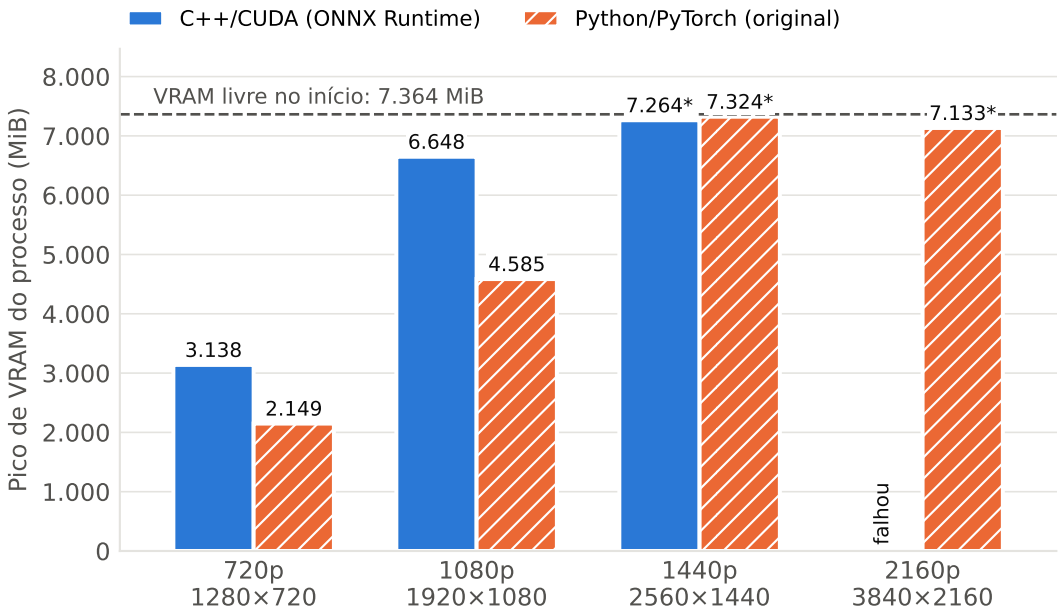


Fonte: Elaborado pelo autor (2026)

5.2 Consumo de VRAM

Ao contrário do esperado pela hipótese deste trabalho, a primeira versão do motor C++ consumiu **mais** VRAM que a *baseline* (Figura 21): 3.138 MiB contra 2.149 MiB em 720p e 6.648 MiB contra 4.585 MiB em 1080p, aproximadamente 45% a mais. Duas causas prováveis foram identificadas na análise da implementação e investigadas na terceira versão do motor (Seção 3.5). Primeiro, o motor mantém alocados durante toda a execução os quatro tensores de filtros (cerca de 1,6 GB em 1080p), enquanto o PyTorch libera os tensores intermediários assim que deixam de ser utilizados. Segundo, o ONNX Runtime reserva memória para os tensores intermediários da rede em um alocador próprio (*arena*) que não a devolve entre execuções. Ajustes das opções do *CUDA Execution Provider* relacionadas ao espaço de trabalho do cuDNN não alteraram o consumo de forma perceptível.

Figura 21 – Pico de VRAM do processo em função da resolução.



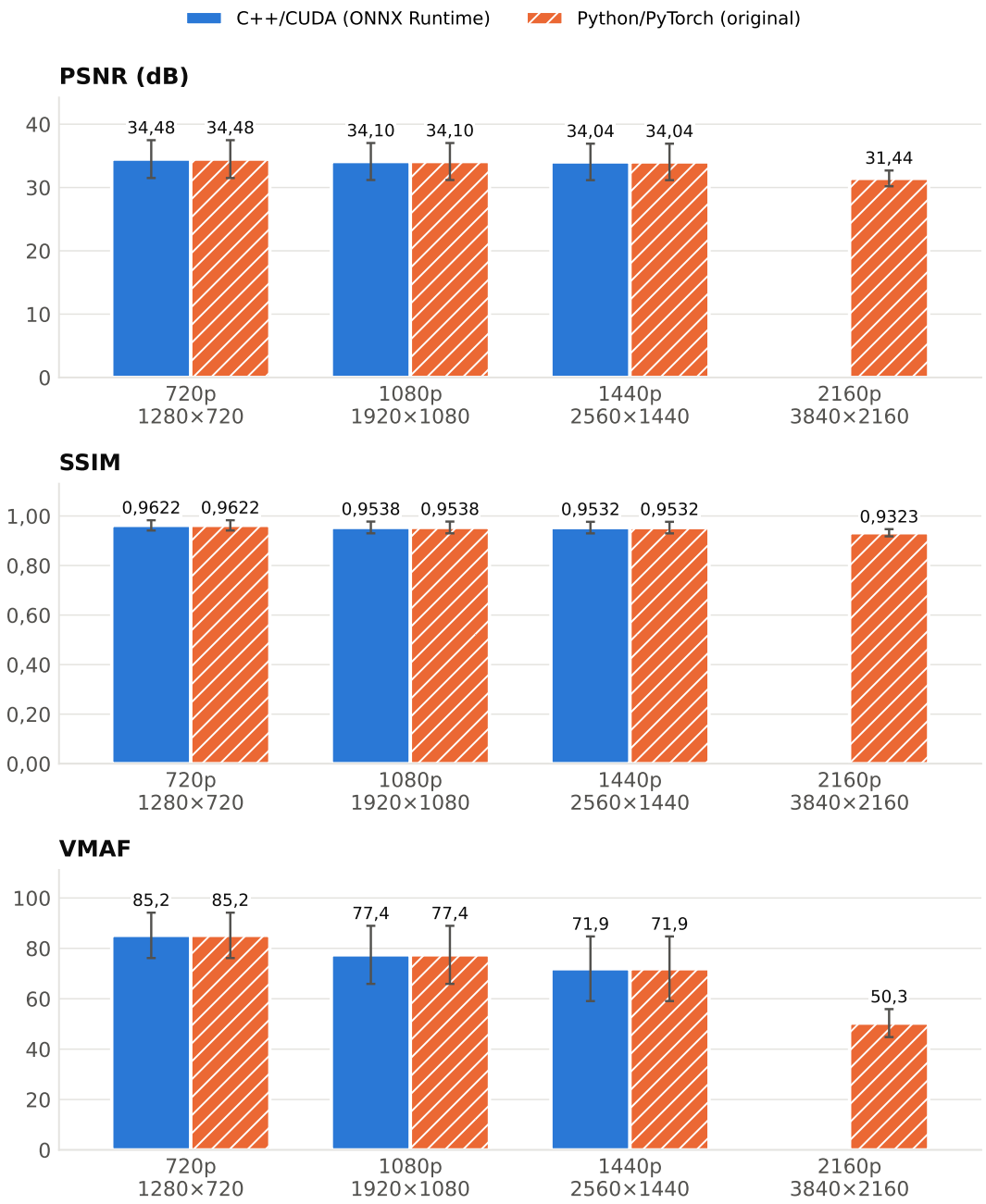
* VRAM saturada: o excedente vai para a memória compartilhada do sistema e não é medido pela NVML; o consumo real é maior que o valor mostrado.

Fonte: Elaborado pelo autor (2026)

5.3 Fidelidade visual

As duas implementações produziram resultados idênticos em todas as resoluções avaliadas: PSNR e SSIM coincidem até a quarta casa decimal, e o VMAF médio difere em menos de 0,001 ponto (Figura 22). Quadro a quadro, a maior diferença de VMAF entre as implementações foi de 0,02 ponto. Esse resultado é coerente com a equivalência verificada na Seção 3.3.4 e confirma que a mudança da forma de execução não degradou a qualidade dos quadros sintetizados.

Figura 22 – Fidelidade visual dos quadros sintetizados: PSNR, SSIM e VMAF médios por resolução, com o desvio padrão entre quadros.



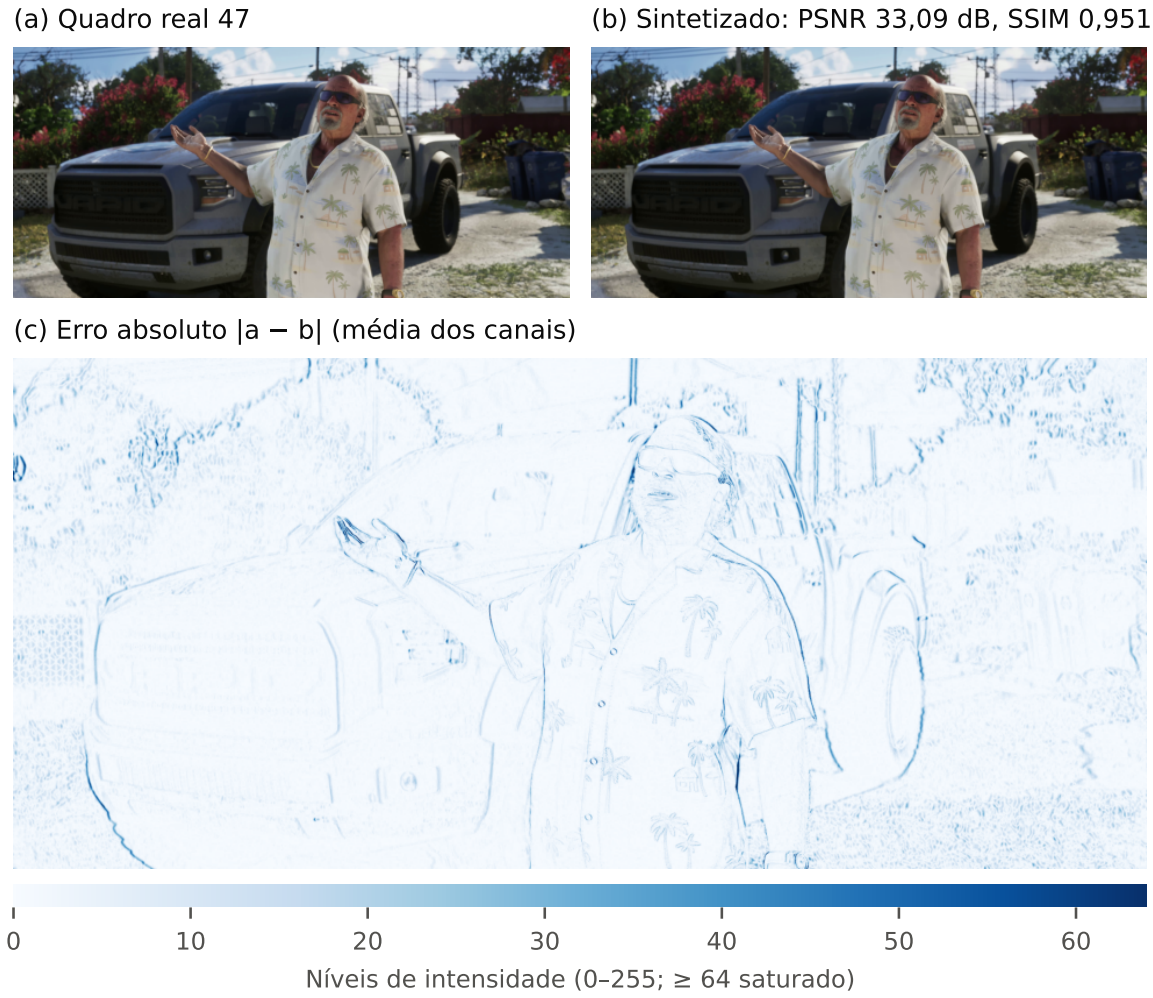
Fonte: Elaborado pelo autor (2026)

Observa-se, por outro lado, uma queda da qualidade com o aumento da resolução. Como em 2160p foram avaliados apenas os quatro primeiros quadros sintetizados, a comparação justa entre resoluções deve considerar os mesmos quadros: nesse recorte, o PSNR cai de 32,48 dB (720p) para 31,44 dB (2160p), uma redução de cerca de 1 dB, enquanto o VMAF cai de 78,88 para 50,33. Duas hipóteses, ainda não verificadas, podem explicar o comportamento. A primeira é o alcance limitado dos filtros: com $N = 51$, o modelo representa deslocamentos de até 25 pixels em cada direção, e o mesmo movimento da

cena ocupa, em 2160p, três vezes mais pixels que em 720p — limitação que motivou propostas como a convolução separável multinível de Wijma et al. (2021), voltada a grandes deslocamentos. A segunda é metodológica e afeta especificamente o VMAF: o modelo utilizado foi calibrado para a visualização de conteúdo em 1080p, e há um modelo específico para 4K.

A Figura 23 apresenta um exemplo qualitativo em 1080p. Para evitar a escolha de um caso favorável, foi utilizado o quadro sintetizado de PSNR mediano, o de índice 47 no trecho de 30 FPS (os quadros de índice ímpar são os sintetizados, Seção 4.4). O mapa de erro mostra que as diferenças em relação ao quadro real se concentram nas bordas de objetos em movimento — como o contorno do personagem e de seu braço — e em texturas finas, enquanto as regiões estáticas são reconstruídas com erro próximo de zero.

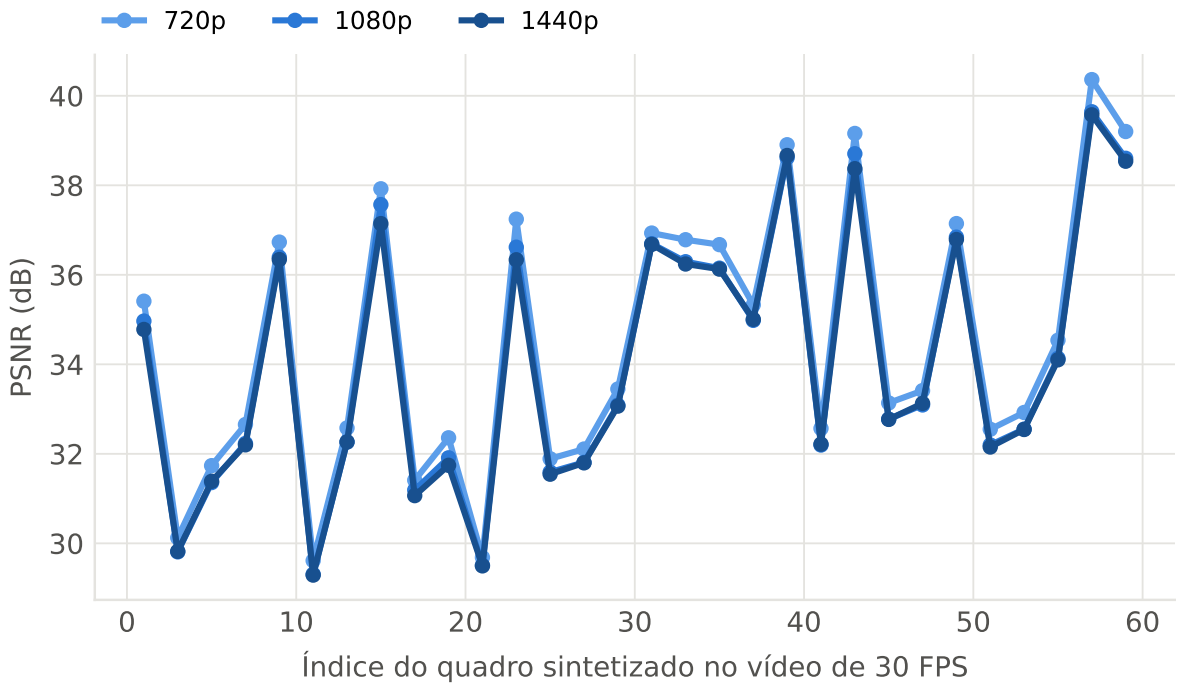
Figura 23 – Exemplo de quadro sintetizado em 1080p (quadro 47, de PSNR mediano): (a) quadro real; (b) quadro sintetizado pelo motor C++; (c) mapa do erro absoluto.



Fonte: Elaborado pelo autor (2026), a partir de quadros do trailer de Rockstar Games (2025). Exibida apenas a área avaliada, sem as tarjas pretas do vídeo original.

A Figura 24 detalha o PSNR de cada quadro sintetizado. As curvas das três resoluções têm o mesmo formato, o que indica que a dificuldade da interpolação é determinada principalmente pelo conteúdo de cada instante (intensidade do movimento), e não pela resolução.

Figura 24 – PSNR de cada quadro sintetizado pelo motor C++, por resolução.



Fonte: Elaborado pelo autor (2026)

5.4 Segunda versão: processamento em blocos

A segunda versão do motor (Seção 3.4) foi avaliada em uma nova bateria, com o mesmo protocolo e o mesmo trecho de vídeo, para blocos de 1024, 512 e 256 pixels e margem de 96 pixels. Para que todas as condições fossem medidas nas mesmas circunstâncias, a bateria repetiu também o motor com o quadro inteiro e a *baseline*. Nas execuções sem saturação da VRAM, as médias repetidas diferiram das da primeira bateria em menos de 0,5%; nas execuções saturadas, em até 4,3%. A Tabela 8 consolida as medições.

Tabela 8 – Desempenho do motor em blocos por resolução (interpolação 2×, margem de 96 pixels)

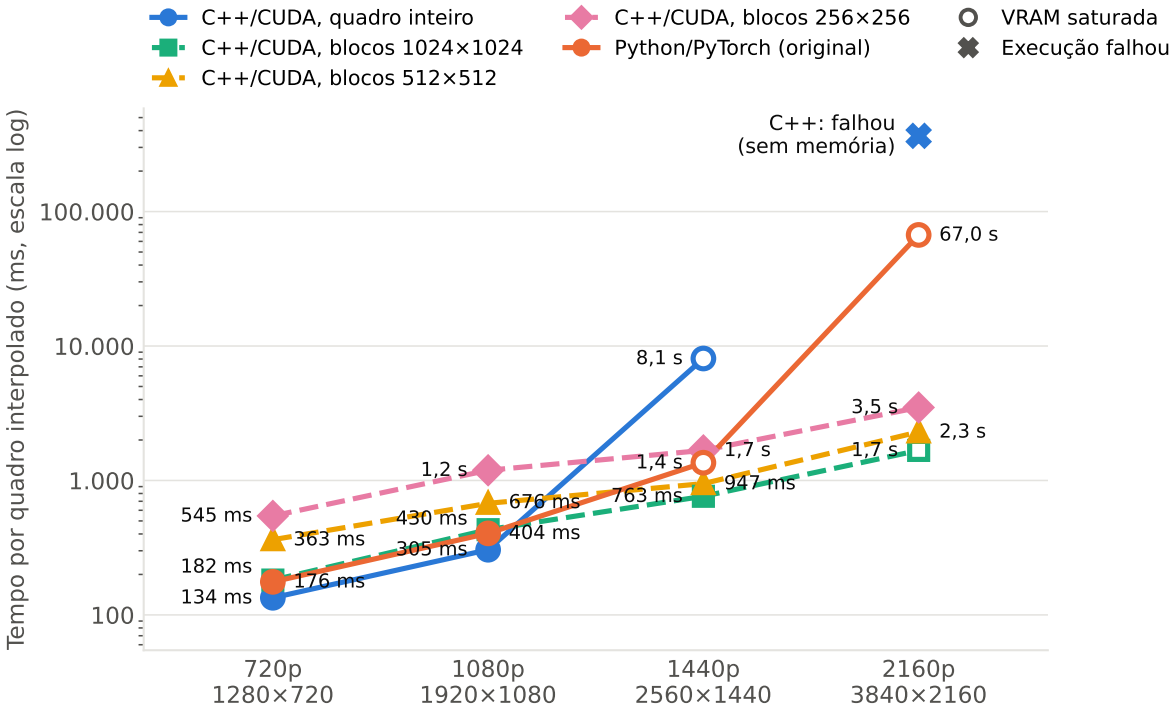
Res.	**Implementação**	**ms/quadro**	**FPS**	**VRAM (MiB)**
720p	C++/CUDA, quadro inteiro	134,1 ± 0,8	7,46	3.138
720p	C++/CUDA, blocos 1024	181,6 ± 8,0	5,51	2.862
720p	C++/CUDA, blocos 512	362,5 ± 19,2	2,76	1.793
720p	C++/CUDA, blocos 256	545,1 ± 24,4	1,83	955
720p	Python/PyTorch	176,2 ± 0,9	5,68	2.155
1080p	C++/CUDA, quadro inteiro	305,3 ± 1,8	3,28	6.650
1080p	C++/CUDA, blocos 1024	430,5 ± 5,4	2,32	4.177
1080p	C++/CUDA, blocos 512	676,0 ± 11,7	1,48	2.183
1080p	C++/CUDA, blocos 256	1.190,4 ± 29,3	0,84	1.000
1080p	Python/PyTorch	404,2 ± 1,8	2,47	4.584
1440p	C++/CUDA, quadro inteiro	8.074,8 ± 10,8*	0,12	7.091*
1440p	C++/CUDA, blocos 1024	762,7 ± 16,0	1,31	5.815
1440p	C++/CUDA, blocos 512	947,4 ± 10,3	1,06	2.253
1440p	C++/CUDA, blocos 256	1.673,7 ± 49,0	0,60	1.078
1440p	Python/PyTorch	1.354,7 ± 10,9*	0,74	7.073*
2160p	C++/CUDA, quadro inteiro	falhou por falta de memória		
2160p	C++/CUDA, blocos 1024	1.673,2 ± 5,0*	0,60	7.403*
2160p	C++/CUDA, blocos 512	2.296,4 ± 5,6	0,44	2.407
2160p	C++/CUDA, blocos 256	3.490,0 ± 5,6	0,29	1.189
2160p	Python/PyTorch	67.031,9 ± 2.189,8*	0,01	7.270*

Latência: média ± desvio padrão (27 medições; três em 2160p). * VRAM saturada: o excedente foi alocado na memória principal e não é contabilizado pela NVML. Fonte: Elaborado pelo autor (2026)

A fidelidade visual não foi afetada pelos blocos. Em todas as resoluções e tamanhos de bloco, as métricas coincidiram com as da *baseline*: a diferença média quadro a quadro foi inferior a 0,001 dB de PSNR e a 10^{-5} de SSIM, e de no máximo 0,007 ponto de VMAF. Por isso, a tabela omite as métricas de qualidade, que são as mesmas da Tabela 7.

A Figura 25 mostra que o efeito dos blocos sobre a latência depende da resolução. Em 720p e 1080p, em que o quadro inteiro cabe na VRAM, os blocos apenas acrescentam tempo: com blocos de 1024 pixels, o motor ficou 35% a 41% mais lento que com o quadro inteiro e 3% a 7% mais lento que a *baseline*; com blocos de 256 pixels, cerca de três vezes mais lento que a *baseline*. A partir de 1440p, a situação se inverte. Nenhuma configuração em blocos saturou a VRAM em 1440p, e o motor passou a ser mais rápido que a *baseline*, que satura: 762,7 ms com blocos de 1024 pixels e 947,4 ms com blocos de 512, contra 1.354,7 ms (reduções de 44% e 30%) e contra 8,1 s do motor com o quadro inteiro. Em 2160p, em que o motor com o quadro inteiro falha, os blocos de 512 pixels sintetizaram cada quadro em 2,3 s, contra 67,0 s da *baseline*. Os blocos de 1024 pixels foram ainda mais rápidos em 2160p (1,7 s), mas saturaram a VRAM, de modo que esse tempo pode ter sido afetado pelo uso da memória principal.

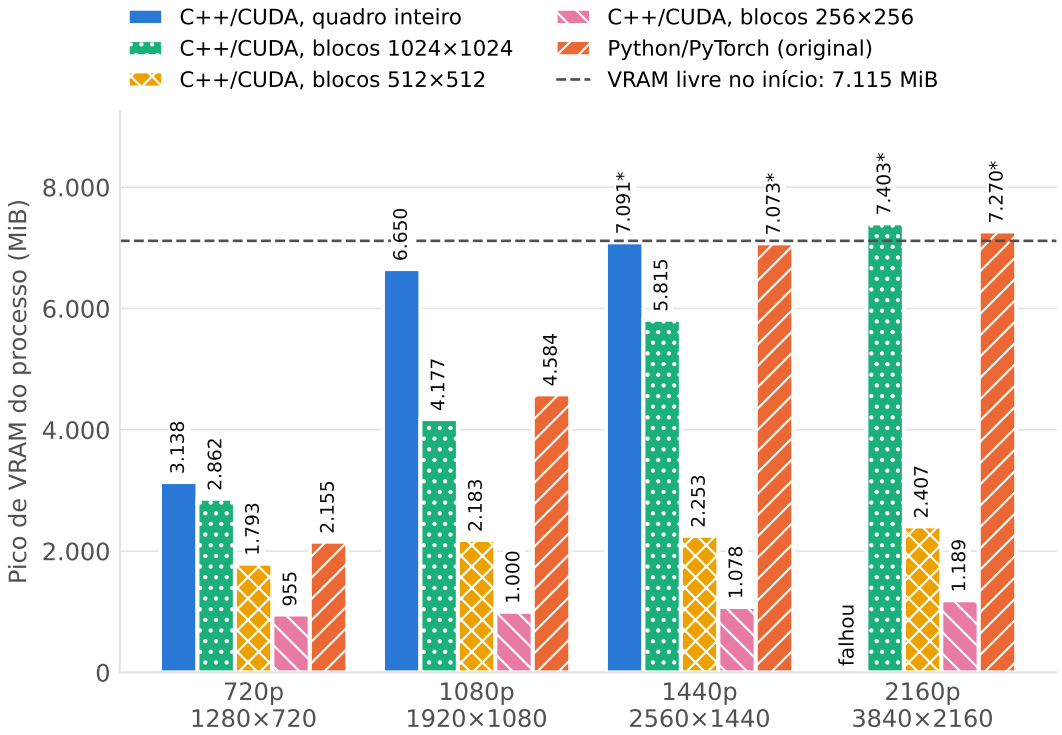
Figura 25 – Tempo médio por quadro interpolado do motor em blocos, comparado ao motor com o quadro inteiro e à *baseline* (escala logarítmica).



Fonte: Elaborado pelo autor (2026). Marcadores vazados indicam VRAM saturada.

Quanto à memória (Figura 26), o pico passa a depender principalmente do tamanho do bloco, e não da resolução: com blocos de 512 pixels, variou de 1.793 MiB (720p) a 2.407 MiB (2160p), e com blocos de 256 pixels, de 955 MiB a 1.189 MiB. A parcela que ainda cresce com a resolução inclui os quadros de entrada e de saída, mantidos inteiros na VRAM (cerca de 250 MB em 2160p), e, possivelmente, a memória que o ONNX Runtime reserva para recortes de formatos diferentes nas bordas do quadro e não devolve. Com blocos de 1024 pixels, o recorte com margem tem 1.216 pixels de lado, e o pico ainda cresce com a resolução até saturar a VRAM em 2160p.

Figura 26 – Pico de VRAM do processo do motor em blocos, comparado ao motor com o quadro inteiro e à *baseline*.



* VRAM saturada: o excedente vai para a memória compartilhada do sistema e não é medido pela NVML; o consumo real é maior que o valor mostrado.

Fonte: Elaborado pelo autor (2026)

O custo em latência dos blocos tem duas origens, separadas na Tabela 9. A principal é a redundância das margens: como cada recorte inclui 96 pixels de cada lado, a rede processa, somados todos os blocos, de 1,15 a 2,9 vezes a área do quadro, conforme a resolução e o tamanho do bloco. A segunda é a eficiência por pixel: em 1080p, o tempo de inferência por megapixel processado sobe de 131,7 ms no quadro inteiro para 152,7 a 194,9 ms nos blocos, possivelmente porque recortes menores ocupam menos a GPU e porque os blocos da borda do quadro têm formatos diferentes. Em 2160p, em que a maior parte dos blocos tem o formato completo, a eficiência por pixel fica próxima da do quadro inteiro (137 a 142 ms por megapixel). O cálculo das estatísticas globais e os recortes dos blocos respondem por menos de 0,4% do tempo por quadro.

Tabela 9 – Área processada pela rede e tempo de inferência no motor em blocos

Res.	**Configuração**	**Recortes**	**Área processada**	**Inferência (ms)**	**ms por megapixel**
1080p	Quadro inteiro	1	1,00×	273,1	131,7
1080p	Blocos 1024	4	1,25×	397,4	152,7
1080p	Blocos 512	12	1,71×	641,0	180,3
1080p	Blocos 256	40	2,85×	1.149,9	194,9
2160p	Blocos 1024	12	1,35×	1.542,1*	137,3
2160p	Blocos 512	40	1,83×	2.156,1	142,1
2160p	Blocos 256	135	2,91×	3.332,4	138,1

Área processada: soma das áreas dos recortes (bloco com margem) em relação à área do quadro. Ms por megapixel: tempo de inferência dividido pela área processada. * VRAM saturada. Fonte: Elaborado pelo autor (2026)

5.5 Terceira versão: alocador ajustado e meia precisão

A terceira versão do motor (Seção 3.5) foi avaliada em uma terceira bateria, com o mesmo protocolo, em cinco séries: o motor em FP32 com o quadro inteiro, agora com o alocador ajustado; o motor em FP16 com o quadro inteiro e com blocos de 1024 e de 512 pixels; e a *baseline*. As medições repetidas da *baseline* diferiram das da segunda bateria em menos de 1%. A Tabela 10 consolida os resultados.

Tabela 10 – Desempenho do motor com o alocador ajustado e em meia precisão (interpolação 2×)

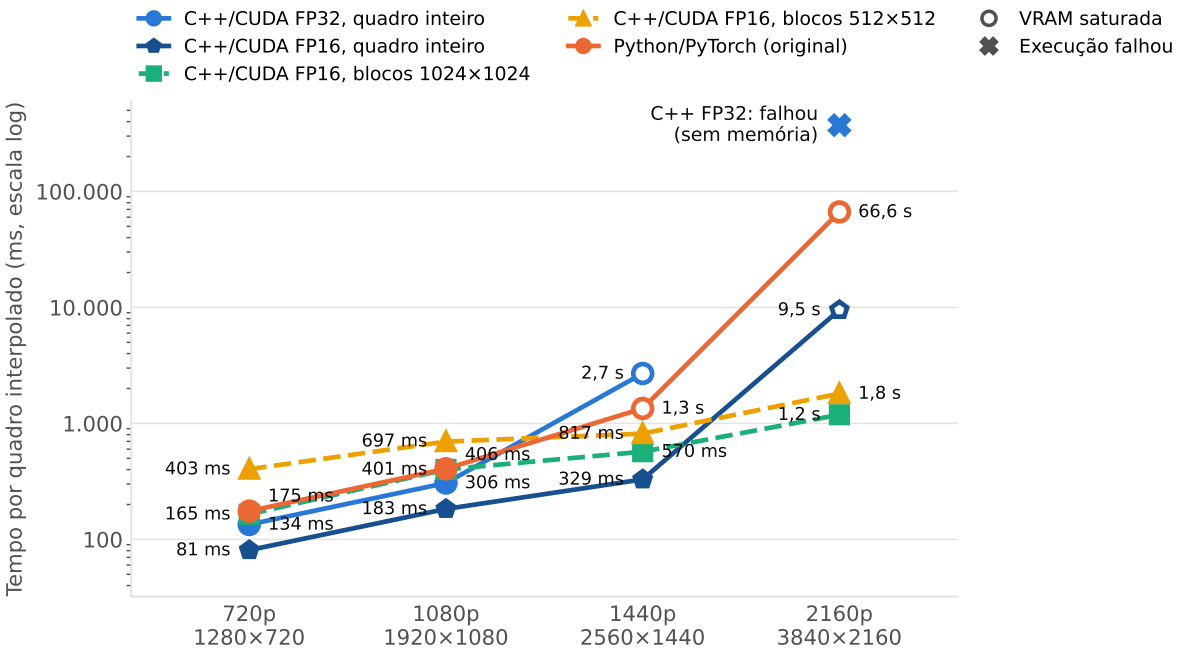
Res.	**Implementação**	**ms/quadro**	**FPS**	**VRAM (MiB)**
720p	C++/CUDA FP32, quadro inteiro	134,4 ± 0,9	7,44	2.944
720p	C++/CUDA FP16, quadro inteiro	80,8 ± 0,9	12,38	1.554
720p	C++/CUDA FP16, blocos 1024	164,7 ± 5,6	6,07	1.568
720p	C++/CUDA FP16, blocos 512	403,0 ± 11,5	2,48	893
720p	Python/PyTorch	175,3 ± 0,8	5,70	2.149
1080p	C++/CUDA FP32, quadro inteiro	306,2 ± 1,2	3,27	4.887
1080p	C++/CUDA FP16, quadro inteiro	183,0 ± 0,6	5,46	3.049
1080p	C++/CUDA FP16, blocos 1024	401,0 ± 24,9	2,49	2.855
1080p	C++/CUDA FP16, blocos 512	696,8 ± 37,3	1,44	920
1080p	Python/PyTorch	406,3 ± 1,7	2,46	4.582
1440p	C++/CUDA FP32, quadro inteiro	2.694,7 ± 9,7*	0,37	7.149*
1440p	C++/CUDA FP16, quadro inteiro	329,0 ± 0,8	3,04	4.800
1440p	C++/CUDA FP16, blocos 1024	570,3 ± 5,0	1,75	2.915
1440p	C++/CUDA FP16, blocos 512	816,7 ± 8,9	1,22	979
1440p	Python/PyTorch	1.345,8 ± 3,7*	0,74	7.204*
2160p	C++/CUDA FP32, quadro inteiro	falhou por falta de memória		
2160p	C++/CUDA FP16, quadro inteiro	9.502,7 ± 43,8*	0,11	7.370*
2160p	C++/CUDA FP16, blocos 1024	1.187,7 ± 2,2	0,84	3.061
2160p	C++/CUDA FP16, blocos 512	1.808,1 ± 12,7	0,55	1.127
2160p	Python/PyTorch	66.594,4 ± 1.768,6*	0,02	7.357*

Latência: média ± desvio padrão (27 medições; três em 2160p). Blocos com margem de 96 pixels. * VRAM saturada: o excedente foi alocado na memória principal e não é contabilizado pela NVML. Fonte: Elaborado pelo autor (2026)

A meia precisão não alterou a fidelidade visual. Em todas as resoluções, a diferença média quadro a quadro em relação à *baseline* foi inferior a 0,001 dB de PSNR e a 10^{-5} de SSIM; no VMAF, foi de no máximo 0,013 ponto de 720p a 1440p e de 0,063 ponto em 2160p, em que apenas quatro quadros são avaliados.

Quanto à latência (Figura 27), a meia precisão reduziu em cerca de 40% o tempo do motor com o quadro inteiro (de 134,4 ms para 80,8 ms em 720p e de 306,2 ms para 183,0 ms em 1080p), o que corresponde a reduções de 53,9% e 55,0% em relação à *baseline*. Em 1440p, o motor em FP16 com o quadro inteiro não saturou a VRAM e sintetizou cada quadro em 329,0 ms, 4,1 vezes menos que a *baseline* (1.345,8 ms, com a VRAM saturada). O motor em FP32 com o quadro inteiro continuou saturando a VRAM em 1440p, mas, com o alocador ajustado, levou 2,7 s por quadro, contra 8,1 s na segunda bateria. Em 2160p, o quadro inteiro satura a VRAM mesmo em FP16 (9,5 s por quadro), e a configuração mais rápida foi a de blocos de 1024 pixels em FP16, com 1,19 s por quadro, contra 66,6 s da *baseline*. Nos blocos de 512 pixels, a meia precisão só reduziu o tempo a partir de 1440p: em 1080p, o motor em FP16 levou 696,8 ms, ante 676,0 ms em FP32 na segunda bateria.

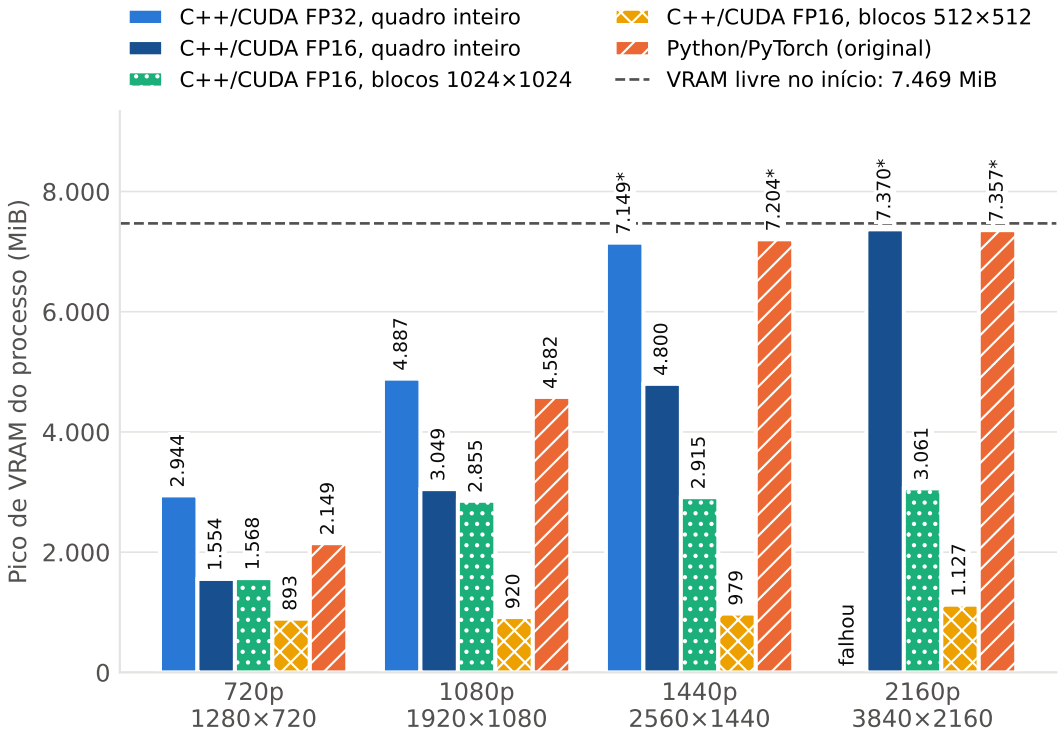
Figura 27 – Tempo médio por quadro interpolado do motor com o alocador ajustado e em meia precisão, comparado à *baseline* (escala logarítmica).



Fonte: Elaborado pelo autor (2026). Marcadores vazados indicam VRAM saturada.

Quanto à memória (Figura 28), o motor em FP16 com o quadro inteiro consumiu 1.554 MiB em 720p e 3.049 MiB em 1080p, 72% e 67% do consumo da *baseline*. Em FP32, com o alocador ajustado, o motor com o quadro inteiro passou a consumir 2.944 MiB e 4.887 MiB nessas resoluções, contra 3.138 MiB e 6.650 MiB na segunda bateria, mas ainda acima da *baseline* (137% e 107%). Com blocos de 512 pixels em FP16, o pico ficou entre 893 MiB e 1.127 MiB em todas as resoluções, inclusive em 2160p.

Figura 28 – Pico de VRAM do processo do motor com o alocador ajustado e em meia precisão, comparado à *baseline*.



* VRAM saturada: o excedente vai para a memória compartilhada do sistema e não é medido pela NVML; o consumo real é maior que o valor mostrado.

Fonte: Elaborado pelo autor (2026)

5.6 Forma de execução e precisão: as quatro combinações

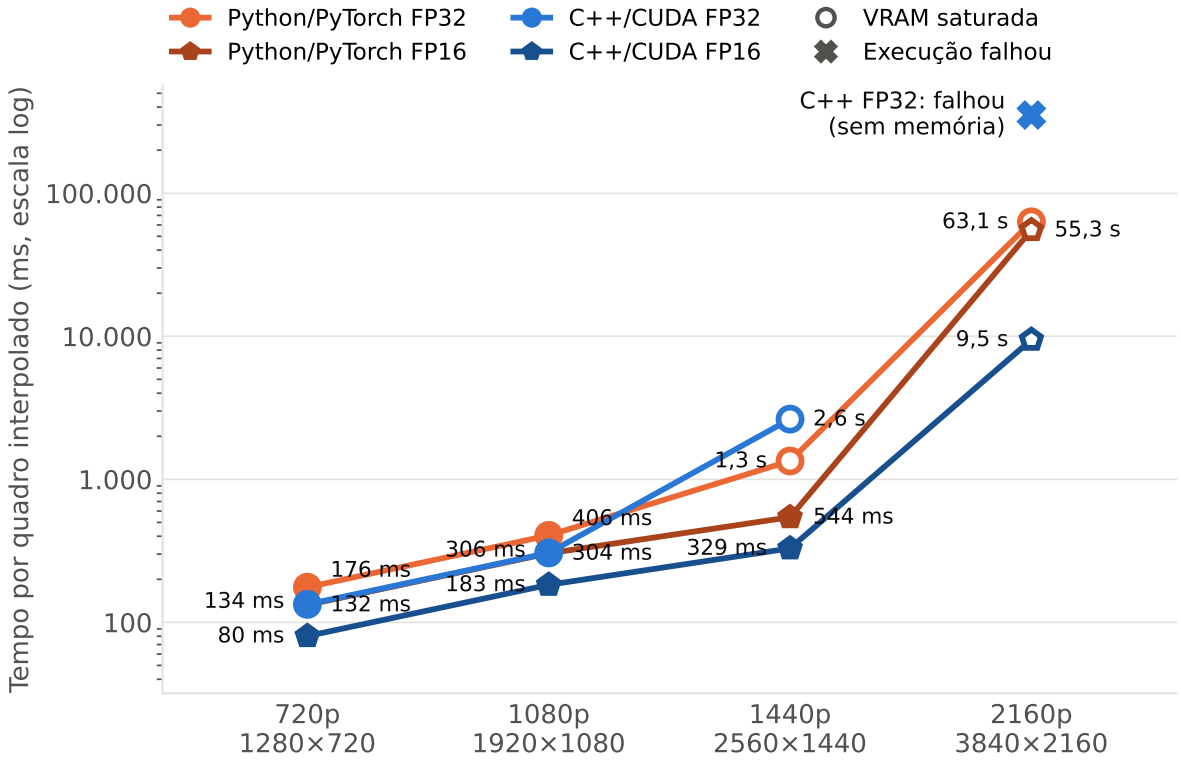
A comparação entre o motor em FP16 e a *baseline* em FP32 reúne dois efeitos. Para separá-los, uma quarta bateria mediu, com o quadro inteiro e o mesmo protocolo, as quatro combinações de implementação e precisão (Tabela 11 e Figura 29). As séries já medidas na terceira bateria repetiram os tempos com diferença inferior a 1% em 720p, 1080p e 1440p sem saturação, mesmo com a GPU mais ocupada no início das execuções (até 1,3 GB, contra 0,5 GB a 0,9 GB).

Tabela 11 – Latência e pico de VRAM nas quatro combinações de implementação e precisão, com o quadro inteiro

	Python FP32		**Python FP16**		**C++ FP32**		**C++ FP16**	
Res.	**ms**	**MiB**	**ms**	**MiB**	**ms**	**MiB**	**ms**	**MiB**
720p	176,0	2.146	132,3	1.837	133,6	2.940	80,2	1.555
1080p	405,5	4.585	303,9	3.801	306,0	4.888	183,0	3.057
1440p	1.343,7*	6.721*	543,6	6.583	2.635,3*	7.231*	329,0	4.771
2160p	63.091,4*	7.345*	55.297,1*	7.377*	falhou		9.463,9*	7.342*

Média de 27 medições (três em 2160p). * VRAM saturada. O C++ FP32 em 1440p atingiu 96,5% da capacidade, abaixo do limiar de 97%, mas foi considerado saturado pelo tempo: a inferência levou 2.576 ms, como na bateria anterior, em que a mesma condição foi sinalizada, e a escala a partir de 1080p indicaria cerca de 545 ms. Fonte: Elaborado pelo autor (2026)

Figura 29 – Tempo médio por quadro nas quatro combinações de implementação e precisão, com o quadro inteiro (escala logarítmica).



Fonte: Elaborado pelo autor (2026). Marcadores vazados indicam VRAM saturada.

A Tabela 12 separa os efeitos nas resoluções sem saturação. Os dois fatores reduzem a latência, e nenhum deles explica sozinho o ganho total de cerca de 55%. O motor mantém a vantagem sobre o PyTorch nas duas precisões, de cerca de 24% em FP32 e de cerca de 40% em FP16. A meia precisão, por sua vez, reduz o tempo nas duas implementações, em cerca

de 25% no PyTorch e 40% no motor. Com isso, o PyTorch em FP16 alcança o tempo do motor em FP32 (303,9 ms contra 306,0 ms em 1080p). A diferença entre os ganhos relativos não foi investigada; uma das explicações possíveis é que o `torch.autocast` mantém os pesos em FP32 e os converte durante a execução.

Tabela 12 – Efeito da forma de execução e da precisão sobre a latência e o pico de VRAM

	Latência		**VRAM (fração)**	
Comparação	**720p**	**1080p**	**720p**	**1080p**
C++ × Python, em FP32	−24,1%	−24,6%	137%	107%
C++ × Python, em FP16	−39,4%	−39,8%	85%	80%
FP16 × FP32, no Python	−24,8%	−25,1%	86%	83%
FP16 × FP32, no C++	−40,0%	−40,2%	53%	63%
C++ FP16 × Python FP32	−54,4%	−54,9%	72%	67%

Todas as reduções de latência são significativas pelo teste t de Welch unilateral ($p < 0{,}001$). VRAM: pico do primeiro termo como fração do pico do segundo. Fonte: Elaborado pelo autor (2026)

Quanto à memória, o resultado se inverte conforme a precisão: em FP32, o motor consome mais que o PyTorch (107% em 1080p); em FP16, consome menos (80%). Parte dessa diferença decorre de uma decisão de implementação: no motor, os filtros permanecem em FP16 na VRAM, enquanto a função de síntese original os converte para FP32. Essa parcela não foi isolada. A fidelidade também não foi afetada na *baseline* em FP16: a diferença média para a *baseline* em FP32 foi inferior a 0,001 dB de PSNR e a 10^{-5} de SSIM, e de no máximo 0,014 ponto de VMAF.

Nas resoluções em que há saturação, os resultados refletem principalmente a memória disponível. Em 1440p, o PyTorch em FP16 deixa de saturar a VRAM e leva 543,6 ms, contra 1.343,7 ms em FP32; o motor em FP16 ainda é 39,5% mais rápido, com 72,5% da sua VRAM. Em 2160p, as três execuções concluídas saturam a VRAM, e a redução de 12,4% do PyTorch em FP16 não é significativa ($p = 0{,}079$, com três medições).

5.7 Conjuntos de referência

A Tabela 13 apresenta a fidelidade visual no Middlebury e na amostra do Vimeo-90K (Seção 4.3). As três implementações avaliadas produziram resultados equivalentes: par a par, a diferença média de PSNR em relação à *baseline* foi de no máximo 0,001 dB, com maior diferença de 0,027 dB em um único par, e a diferença média de SSIM foi inferior a 2×10^{-5}.

Tabela 13 – Fidelidade visual nos conjuntos de referência

	PSNR (dB)		**SSIM**	
Implementação	**Middlebury**	**Vimeo-90K**	**Middlebury**	**Vimeo-90K**
C++/CUDA FP32	37,22	34,46	0,9680	0,9590
C++/CUDA FP16	37,22	34,46	0,9679	0,9590
Python/PyTorch	37,22	34,46	0,9680	0,9590

Middlebury: 12 sequências. Vimeo-90K: 200 trios sorteados do conjunto de teste. Fonte: Elaborado pelo autor (2026)

O *script* de avaliação distribuído com o código de Niklaus, Mai e Wang (2021), executado no mesmo ambiente, resulta em 37,28 dB e 0,9681 no Middlebury. A diferença de 0,06 dB decorre apenas da conversão para 8 bits: o *script* arredonda a saída da rede, enquanto o motor e o código de interpolação original a truncam. Esses conjuntos confirmam, portanto, em imagens de referência da literatura, a equivalência observada no vídeo do GTA VI.

5.8 Discussão frente à hipótese

A Tabela 14 resume a avaliação dos três critérios da Seção 4.6 nas resoluções em que a VRAM foi suficiente para as duas implementações.

Tabela 14 – Avaliação dos critérios da hipótese em 720p e 1080p

Critério	**720p**	**1080p**	**Resultado**
H1 – Latência	$-24{,}3\%$; $t = 201$, $p < 0{,}001$	$-24{,}6\%$; $t = 182$, $p < 0{,}001$	Atendido
H2 – VRAM	+46% (3.138 contra 2.149 MiB)	+45% (6.648 contra 4.585 MiB)	Não atendido
H3 – Fidelidade	$\lvert\Delta\rvert$ médio: PSNR $< 0{,}0001$ dB; SSIM $< 10^{-6}$; VMAF 0,003	$\lvert\Delta\rvert$ médio: PSNR $< 0{,}0001$ dB; SSIM 0; VMAF $< 0{,}001$	Atendido

Variações do motor C++ em relação à *baseline*. Teste t de Welch unilateral com 27 medições por implementação. Fonte: Elaborado pelo autor (2026)

Na primeira versão do motor, a hipótese foi, portanto, **confirmada apenas parcialmente**. A redução de latência foi observada nas resoluções em que a VRAM é suficiente, com margem muito superior ao limiar de 10% e sem qualquer perda de fidelidade visual. A redução do consumo de VRAM, porém, não se verificou: a simples transposição da inferência para C++ e ONNX Runtime, mantendo o processamento do quadro inteiro, consome mais memória que a implementação original e degrada mais rapidamente quando a memória se esgota. Em 1440p e 2160p, nenhum dos critérios de desempenho é atendido,

pois as duas implementações saturam a VRAM e o motor C++ fica mais lento que a *baseline*.

Esse resultado indica que o benefício esperado sobre a memória não decorre automaticamente da troca de linguagem ou de *framework*, mas depende de um controle explícito do uso de memória — em particular, de não manter simultaneamente na VRAM os tensores de filtros e as ativações intermediárias do quadro inteiro. Indica também uma distinção que a formulação inicial da hipótese não fazia: a separabilidade dos filtros é o que torna o modelo executável em 1080p em uma GPU de 8 GB (1,6 GB de filtros contra os 40 GB de um *kernel* bidimensional), mas, como está presente nas duas implementações, ela não explica as diferenças entre elas. Essas diferenças decorrem da forma de execução.

A segunda versão do motor introduz esse controle explícito, e a Tabela 15 aplica a ela os mesmos critérios, nas mesmas resoluções. O critério de fidelidade (H3) é atendido por todos os tamanhos de bloco (Seção 5.4).

Tabela 15 – Avaliação dos critérios da hipótese para o motor em blocos, em 720p e 1080p

Bloco	**Latência (720p / 1080p)**	**VRAM (720p / 1080p)**	**H1**	**H2**
1024 pixels	+3,1% / +6,5%	133% / 91%	Não atendido	Não atendido
512 pixels	+106% / +67%	83% / 48%	Não atendido	Atendido
256 pixels	+209% / +195%	44% / 22%	Não atendido	Atendido

Latência: variação em relação à *baseline*. VRAM: pico do motor como fração do pico da *baseline* (limiar de 90%); nenhuma das execuções saturou a VRAM. Medições da segunda bateria. Fonte: Elaborado pelo autor (2026)

Nas resoluções em que os critérios foram definidos, nenhuma configuração atende H1 e H2 ao mesmo tempo. Os blocos de 512 e de 256 pixels cumprem o critério de memória, mas com latências maiores que as da *baseline*; os blocos de 1024 pixels ficam a um ponto percentual do limiar de VRAM em 1080p e também são mais lentos que a *baseline*. O compromisso entre memória e latência é, portanto, o resultado central da segunda versão: o motor com o quadro inteiro é a opção mais rápida enquanto a VRAM é suficiente, e o processamento em blocos é o que mantém o motor utilizável quando ela não é. Esse compromisso decorre principalmente do reprocessamento das margens (Tabela 9), e não de uma ineficiência da execução em C++.

Em 1440p e 2160p, fora do alcance dos critérios porque a *baseline* satura a VRAM, o quadro se inverte. Com blocos de 512 ou de 256 pixels, o motor processou todas as resoluções sem saturar a VRAM, e os blocos de 1024 e de 512 pixels foram mais rápidos que a *baseline* em 1440p ($-43{,}7\%$ e $-30{,}1\%$; teste t de Welch unilateral, $p < 0{,}001$). Em

2160p, a diferença também é significativa ($p < 0{,}001$), mas foi estimada com apenas três medições por condição e contra uma *baseline* que opera em grande parte na memória principal. Nessas resoluções, a vantagem do motor em blocos decorre de evitar a saturação, e não de uma execução mais rápida da mesma quantidade de trabalho.

A terceira versão atua sobre a memória sem reprocessar regiões do quadro, e a Tabela 16 aplica a ela os mesmos critérios. O critério de fidelidade (H3) é atendido por todas as configurações (Seções 5.5 e 5.7).

Tabela 16 – Avaliação dos critérios da hipótese para a terceira versão do motor, em 720p e 1080p

Configuração	**Latência (720p / 1080p)**	**VRAM (720p / 1080p)**	**H1**	**H2**
FP32, quadro inteiro	−23,3% / −24,6%	137% / 107%	Atendido	Não atendido
FP16, quadro inteiro	−53,9% / −55,0%	72% / 67%	Atendido	Atendido
FP16, blocos 1024	−6,0% / −1,3%	73% / 62%	Não atendido	Atendido
FP16, blocos 512	+130% / +72%	42% / 20%	Não atendido	Atendido

Latência: variação em relação à *baseline*; as reduções do quadro inteiro são significativas pelo teste t de Welch ($p < 0{,}001$), e a redução com blocos de 1024 pixels não atinge o limiar de 10% (e, em 1080p, não é significativa, $p = 0{,}14$). VRAM: pico do motor como fração do pico da *baseline*; nenhuma das execuções saturou a VRAM. Medições da terceira bateria. Fonte: Elaborado pelo autor (2026)

O motor em FP16 com o quadro inteiro é a primeira configuração a atender os três critérios simultaneamente em 720p e em 1080p: reduz a latência em mais da metade e o pico de VRAM em cerca de 30%, com fidelidade equivalente à da implementação original. Esse resultado reúne dois efeitos, separados na Seção 5.6. Parte do ganho vem da precisão, que também acelera o PyTorch: em FP16, ele alcança o tempo do motor em FP32. Mas a forma de execução mantém uma vantagem independente: comparados na mesma precisão, o motor é cerca de 24% mais rápido em FP32 e cerca de 40% em FP16. Os critérios H1 e H2 também seriam atendidos pelo motor em FP16 tomando a *baseline* em FP16 como referência, com redução de 39,8% na latência e pico de 80% em 1080p. Quanto à memória, o resultado não pode ser atribuído à troca de linguagem: em FP32, com o alocador ajustado, o motor ainda consome mais que a *baseline* (107% em 1080p), e a vantagem em FP16 inclui a decisão de manter os filtros em meia precisão também na síntese.

Em 1440p e 2160p, a terceira versão amplia o que a segunda havia mostrado. O motor em FP16 com o quadro inteiro processa 1440p sem saturar a VRAM, 4,1 vezes mais rápido

que a *baseline*, e o motor em FP16 com blocos de 1024 pixels processa 2160p com 3,1 GB, em 1,19 s por quadro. Em síntese, os resultados indicam que a redução de latência decorre da forma de execução — presente desde a primeira versão, em FP32 — e é ampliada pela precisão numérica, enquanto a redução de memória decorre de decisões explícitas de gerenciamento — desativar o padrão de memória do ONNX Runtime, reduzir a precisão e, nas resoluções em que o quadro inteiro não cabe, processar em blocos.

6 Conclusão

Este trabalho investigou em que medida um motor de inferência nativo em C++ e CUDA para o modelo SepConv é capaz de reduzir a latência e o consumo de VRAM em uma GPU de consumo de 8 GB, mantendo a fidelidade visual da implementação original (Seção 1.2). Para isso, foi desenvolvido um motor que integra a leitura de vídeo pela OpenCV, a inferência da rede SepConv++ pelo ONNX Runtime com *I/O binding* e um *kernel* CUDA próprio para a síntese do quadro, numericamente equivalente à implementação original em PyTorch (93,67 dB de PSNR entre as saídas). A exportação do modelo para ONNX foi corrigida para aceitar resoluções arbitrárias e validada contra a rede original.

Na primeira versão do motor, os resultados dependeram da métrica considerada. Quanto à **latência**, o motor C++ reduziu o tempo por quadro em cerca de 24% em 720p e 1080p, de 404,9 ms para 305,5 ms em 1080p, com quadros de fidelidade idêntica à da implementação original. Quanto ao **consumo de VRAM**, a primeira versão do motor não trouxe redução: consumiu cerca de 45% mais memória que a *baseline* e, a partir de 1440p, ficou mais lento que ela, chegando a falhar em 2160p. A transposição para C++ é, portanto, suficiente para reduzir a latência, mas não para reduzir a memória; para isso, é necessário um controle explícito da alocação, que a primeira versão ainda não exerce. Esse controle foi introduzido em uma segunda versão do motor, com processamento em blocos (Seção 3.4), de saída equivalente à do quadro inteiro. Com ela, o pico de VRAM passou a depender do tamanho do bloco — no máximo 2.407 MiB com blocos de 512 pixels, mesmo em 2160p — e o motor processou todas as resoluções sem saturar a VRAM, sendo mais rápido que a *baseline* em 1440p e 2160p. Em 720p e 1080p, porém, os blocos aumentaram a latência, e nenhuma configuração em blocos atendeu aos critérios de latência e de memória ao mesmo tempo: a redução de memória tem como custo o reprocessamento das margens dos blocos.

A terceira versão do motor eliminou esse compromisso nas resoluções em que o quadro inteiro cabe na VRAM. O estudo do alocador mostrou que a maior parte da memória excedente da primeira versão vinha do padrão de memória do ONNX Runtime, cuja desativação reduziu o pico em 1080p em 29% sem custo de tempo; e a inferência em meia precisão, com fidelidade equivalente à original no vídeo de teste, no Middlebury e no Vimeo-90K, reduziu ainda mais a memória e o tempo. Com o quadro inteiro em FP16, o motor sintetizou cada quadro em 183,0 ms em 1080p, contra 406,3 ms da *baseline*, com 67% da sua VRAM, atendendo pela primeira vez aos três critérios da hipótese em 720p e 1080p; em 1440p, processou o quadro inteiro sem saturar a VRAM, 4,1 vezes mais rápido que a *baseline*, e, em 2160p, com blocos de 1024 pixels, em 1,19 s por quadro com 3,1 GB.

A resposta à questão de pesquisa é, portanto, afirmativa, com uma ressalva sobre a sua origem. A redução de latência decorre da forma de execução: está presente desde a primeira

versão, com a mesma precisão da implementação original. A redução de memória, por outro lado, não decorre da troca de linguagem ou de *framework*, mas de decisões explícitas de gerenciamento — a configuração do alocador, a precisão numérica e, quando necessário, o processamento em blocos —, que o motor nativo torna diretamente controláveis. A medição das quatro combinações de implementação e precisão confirmou essa separação: na mesma precisão, o motor é cerca de 24% mais rápido em FP32 e 40% em FP16, e a meia precisão reduz o tempo em cerca de 25% no PyTorch e 40% no motor. O ganho total de 55% resulta, portanto, dos dois fatores.

Como contribuições, destacam-se: a correção da exportação ONNX do modelo para resoluções arbitrárias; um motor de inferência em C++/CUDA verificado contra a implementação original; uma bateria de testes reprodutível, com medição por estágio e critérios de VRAM que consideram o *Sysmem Fallback* do Windows; o processamento em blocos com normalização global e margem de sobreposição definida experimentalmente, com a quantificação do seu compromisso entre memória e latência; a identificação do padrão de memória do ONNX Runtime como principal fonte de consumo excedente; a inferência em meia precisão com filtros em FP16 e acumulação em FP32, sem perda de fidelidade; e a constatação, relatada mesmo sendo contrária à hipótese inicial, de que a troca de linguagem e de *framework* não reduz por si só o consumo de memória.

6.1 Limitações

- As medições de desempenho utilizaram um único trecho de vídeo de 61 quadros, sem fixação dos *clocks* da GPU. A *baseline* e o motor com o quadro inteiro foram executados em mais de uma bateria, com diferenças inferiores a 1% fora da saturação; as configurações em blocos foram executadas uma única vez;
- A *baseline* em FP16 usa o `torch.autocast`, que mantém os pesos em FP32 e devolve os filtros a FP32 na síntese; outra forma de conversão no PyTorch poderia ter desempenho diferente, e a parcela de memória devida aos filtros não foi isolada;
- O limiar de 97% da capacidade não detectou uma execução com sinais claros de saturação (C++ FP32 em 1440p, na quarta bateria, com 96,5%), identificada pelo tempo; o limiar pode, portanto, deixar de sinalizar saturações próximas a ele;
- O Vimeo-90K foi obtido de uma cópia pública, e não do servidor oficial, e apenas 200 dos 3.782 trios do conjunto de teste foram avaliados;
- Em 2160p, apenas quatro quadros sintetizados foram avaliados, e o VMAF utilizou o modelo calibrado para 1080p;
- A *baseline* foi medida apenas pelo tempo total, o que impede atribuir a redução de latência a um estágio específico;

- A condição de controle (ONNX Runtime sem o *kernel* próprio) não foi executada, de modo que não é possível separar o efeito do mecanismo de inferência do efeito do *kernel* de síntese;
- O efeito do padrão de memória do ONNX Runtime foi medido, mas o mecanismo que o explica (a coexistência da memória das primeiras execuções com o bloco planejado) não foi verificado;
- O custo do processamento em blocos foi decomposto pela área processada; as causas da menor eficiência por pixel dos blocos pequenos não foram isoladas;
- No Windows, a NVML não informa o consumo por processo, e o consumo foi estimado pela diferença em relação à linha de base da GPU.

6.2 Trabalhos futuros

1. **Redução do custo do processamento em blocos**: escolher o tamanho do bloco automaticamente a partir da VRAM livre, usando o quadro inteiro sempre que ele couber; usar recortes de formato fixo, para evitar formatos irregulares nas bordas do quadro; e avaliar margens menores, reduzindo a área reprocessada;
2. **Origem do ganho em meia precisão**: converter também os pesos da *baseline* para FP16 e medir as duas implementações por estágio, para explicar por que a meia precisão acelera mais o motor que o PyTorch e isolar a memória dos filtros;
3. **Outros mecanismos de inferência**: comparar o ONNX Runtime com o TensorRT e avaliar a quantização em 8 bits, verificando se a fidelidade se mantém;
4. **Condição de controle e medição por estágio da *baseline***: executar a condição de controle e instrumentar a *baseline* por estágio, para localizar a origem da redução de latência;
5. **Ampliação da base de avaliação**: avaliar o conjunto de teste completo do Vimeo-90K (XUE et al., 2019) e medir o desempenho em mais trechos de vídeo, com mais quadros em 2160p, o modelo VMAF para 4K e repetições das baterias, para fortalecer a validade estatística dos resultados.

Referências

BAI, J. et al. Onnx: Open neural network exchange. *GitHub Repository*, 2019. Disponível em: <https://onnx.ai/>. 25

BAKER, S. et al. A database and evaluation methodology for optical flow. *International Journal of Computer Vision*, v. 92, n. 1, p. 1–31, 2011. 9, 10, 17, 19, 20, 21, 23, 28, 30

BAO, W. et al. Depth-aware video frame interpolation. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [s.n.], 2019. Acesso em: 12 abr. 2026. Disponível em: <https://arxiv.org/abs/1904.00830>. 10

BORDWELL, D.; THOMPSON, K.; SMITH, J. *Film Art: An Introduction*. 11. ed. [S.l.]: McGraw-Hill Education, 2016. 8

GONZALEZ, R. C.; WOODS, R. E. *Processamento Digital de Imagens*. 3. ed. São Paulo: Pearson, 2009. 16, 18, 27

HAN, P. et al. Motion-aware video frame interpolation. *Neural Networks*, v. 178, p. 106433, 2024. ISSN 0893-6080. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0893608024003575>. 32

HENNESSY, J. L.; PATTERSON, D. A. *Computer Architecture: A Quantitative Approach*. 7. ed. [S.l.]: Morgan Kaufmann, 2022. 26, 27, 31

HORE, A. C.; ZIOU, D. Image quality metrics: Psnr vs. ssim. *20th International Conference on Pattern Recognition*, p. 2366–2369, 2010. 29

KIRK, D. B.; HWU, W. mei W. *Programming Massively Parallel Processors: A Hands-on Approach*. 5. ed. [S.l.]: Morgan Kaufmann, 2023. 18, 23, 24, 25, 26, 27, 28, 31

LI, Z. et al. *Toward A Practical Perceptual Video Quality Metric*. [S.l.], 2016. Acesso em: 25 ago. 2026. Disponível em: <https://netflixtechblog.com/toward-a-practical-perceptual-video-quality-metric-653f208b9652>. 30

Linux Foundation AI; Microsoft. *ONNX Runtime Documentation*. 2026. Acesso em: 28 maio 2026. Disponível em: <https://onnxruntime.ai/>. 25

Netflix. *VMAF: Video Multimethod Assessment Fusion*. 2026. Acesso em: 25 ago. 2026. Disponível em: <https://github.com/Netflix/vmaf>. 49

NIKLAUS, S.; MAI, L.; LIU, F. Video frame interpolation via adaptive separable convolution. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [s.n.], 2017. Acesso em: 12 abr. 2026. Disponível em: <https://arxiv.org/abs/1708.01692>. 8, 9, 19, 22, 23, 30, 35

NIKLAUS, S.; MAI, L.; WANG, O. Revisiting adaptive convolutions for video frame interpolation. In: *IEEE Winter Conference on Applications of Computer Vision (WACV)*. [s.n.], 2021. Acesso em: 12 abr. 2026. Disponível em: <https://openaccess.thecvf.com/content/WACV2021/html/Niklaus_Revisiting_Adaptive_Convolutions_for_Video_Frame_Interpolation_WACV_2021_paper.html>. 20, 22, 30, 32, 35, 40, 45, 46, 47, 68

NVIDIA Corporation. *CUDA C++ Programming Guide*. 2025. Acesso em: 15 maio 2026. Disponível em: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>. 24, 27, 28, 31

Rockstar Games. *Grand Theft Auto VI: Trailer 2*. 2025. Acesso em: 25 out. 2026. Disponível em: <https://www.rockstargames.com/VI>. 47, 57

SZELISKI, R. *Computer Vision: Algorithms and Applications*. 2. ed. New York: Springer, 2022. 8, 17, 18, 19, 21

TEED, Z.; DENG, J. Raft: Recurrent all-pairs field transforms for optical flow. In: *European Conference on Computer Vision (ECCV)*. [s.n.], 2020. Acesso em: 12 abr. 2026. Disponível em: <https://arxiv.org/abs/2003.12039>. 8, 21, 33

Valve Corporation. *Steam Hardware & Software Survey*. 2026. Acesso em: 28 maio 2026. Disponível em: <https://store.steampowered.com/hwsurvey>. 12, 26

WANG, Z. et al. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*, IEEE, v. 13, n. 4, p. 600–612, 2004. 29

WATKINSON, J. *The Art of Digital Video*. 4. ed. Oxford: Focal Press, 2009. 8

WIJMA, Q. et al. Multi-level adaptive separable convolution for large-motion video frame interpolation. In: *ICCV Workshops*. [s.n.], 2021. Acesso em: 12 abr. 2026. Disponível em: <https://openaccess.thecvf.com/content/ICCV2021W/PBDL/html/Wijma_Multi-Level_Adaptive_Separable_Convolution_for_Large-Motion_Video_Frame_Interpolation_ICCVW_2021_paper.html>. 57

XUE, T. et al. Video enhancement with task-oriented flow. In: *International Journal of Computer Vision*. [S.l.: s.n.], 2019. v. 127, n. 8, p. 1106–1125. 47, 74